排序

阅读顺序

点标题跳到对应页

插入与希尔排序 第 21 页
看每趟处理的有序前缀或等距子序列。

交换与快速排序 第 46 页
跟踪枢轴位置，分清划分和递归的区间。

选择与堆排序 第 63 页
区分建立堆、交换堆顶和向下筛选。

归并排序 第 83 页
先确认两段分别有序，再跟踪合并。

链式基数排序 第 111 页
从低位到高位分配并收集，桶内保持原顺序。

排序方法比较 第 114 页
按时间、辅助空间和稳定性比较。

易错点

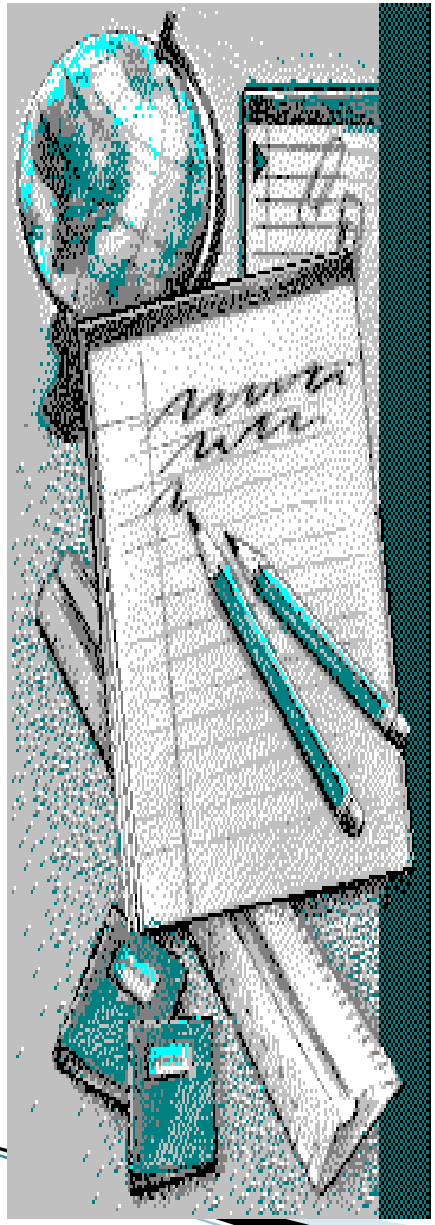
稳定性看相同关键字记录的相对顺序。归并时相等先取左段，才能保留原顺序。

第 57 页的快排区间含端点，两条记录也应参与划分。持续单边划分会退化到平方阶。

随堂自测

对相同关键字的记录 4a、4b 排序，稳定性要求什么？

答案 排序后 4a 仍在 4b 前。



第九章

排序

10.1 概述

10.2 插入排序

10.3 快速排序

10.4 堆排

10.5 归并排序

10.6 基数排序

10.7 各种排序方法的综合比较

10.8 外部排序

10.1 概 述

一、排序的定义

二、内部排序和外部排序

三、内部排序方法的分类



一、什么是排序?

排序是计算机内经常进行的一种操作，其目的是将一组**“无序”的记录序列调整为“有序”**的记录序列。

例如：将下列关键字序列

52, 49, 80, 36, 14, 58, 61, 23, 9

调整为

14, 23, 36, 49, 52, 58, 61 ,75, 8

一般情况下，

假设含n个记录的序列为$\{ R_1, R_2, \ldots, R_n \}$

其相应的关键字序列为 $\{ K_1, K_2, \ldots, K_n \}$

这些关键字相互之间可以进行比较，即在它们之间存在着这样一个关系：

$$K_{p1} \leq K_{p2} \leq \ldots \leq K_{pn}$$

按此固有关系将上式记录序列重新排列为

$$\{ R_{p1}, R_{p2}, \ldots, R_{pn} \}$$

的**操作**称作**排序**。



二、内部排序和外部排序

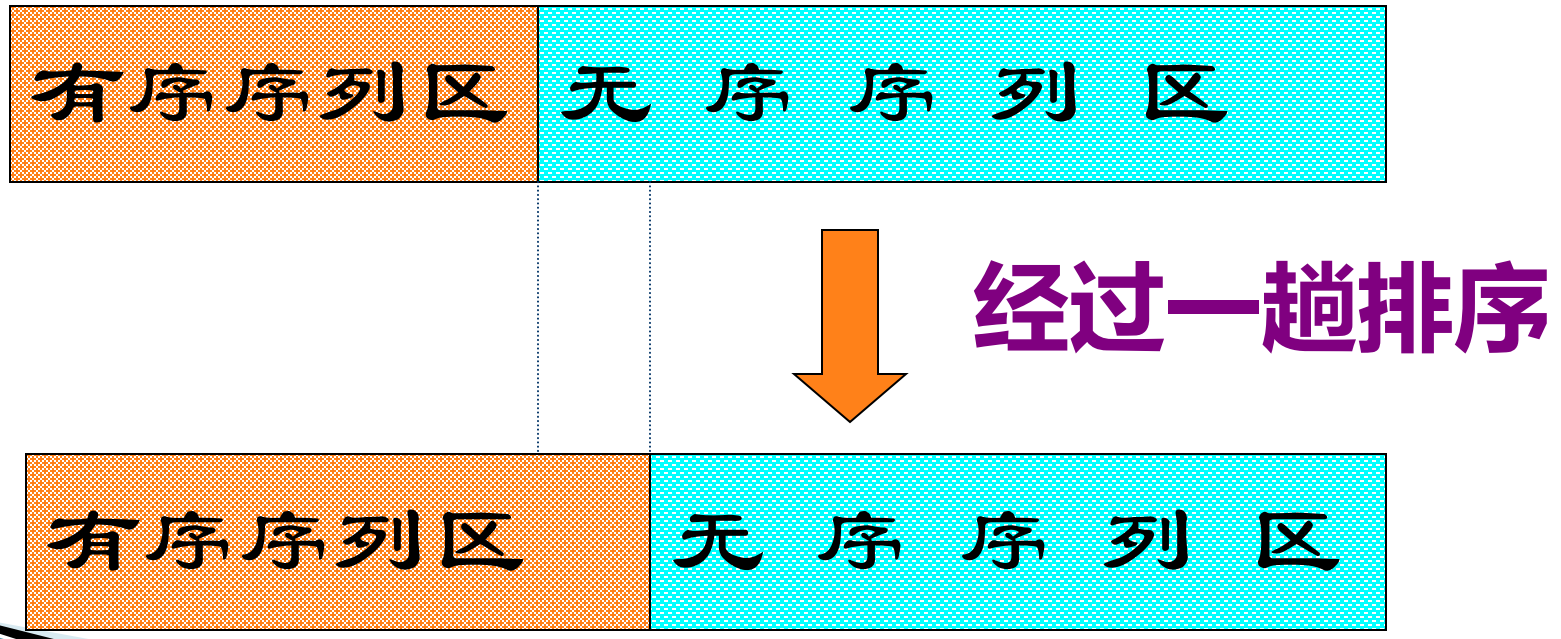
若整个排序过程**不需要访问外存**便能完成，则称此类排序问题为**内部排序**；

反之，若参加排序的记录数量很大，整个序列的排序过程不可能在内存中完成，则称此类排序问题为**外部排序**。



三、内部排序的方法

内部排序的过程是一个**逐步扩大**记录的**有序序列长度**的过程。



基于不同的“**扩大**” 有序序列长度的**方法，内部排序方法**大致可分下列几种类型：

插入类　　交换类　　选择类

归并类　　其它方法

待排记录的数据类型定义如下：

```
#define MAXSIZE 1000                    // 待排顺序表最大长度
typedef  int  KeyType;   // 关键字类型为整数类型

typedef  struct {
    KeyType   key;                  // 关键字项
    InfoType  otherinfo;   // 其它数据项
} RcdType;                          // 记录类型

typedef  struct {
    RcdType    r[MAXSIZE+1]; // r[0]闲置
    int              length;              // 顺序表长度
} SqList;                               // 顺序表类型
```

- 排序算法的稳定性: 如果在对象序列中有两个对象r[i]和r[j]，它们的关键码 k[i] == k[j]，且在排序之前，对象r[i]排在r[j]前面。如果在排序之后，对象r[i]仍在对象r[j]的前面，则称这个排序方法是稳定的，否则称这个排序方法是不稳定的

稳定的排序方法指的是，对于两个关键字相等的记录，它们在序列中的相对位置，在排序之前和排序之后，没有发生改变

排序之前：
{ ... $R_i(K)$ ... $R_j(K)$ ... }
排序之后：
{ ... $R_i(K)$ ... $R_j(K)$ ... }

排序前 (56, 34, 47, 23, 66, 18, 82,

若排序后得到结果

(18, 23, 34, 47, 47, 56, 6

则称该排序方法是**稳定**的

若排序后得到结果

(18, 23, 34, 47, 47, 56, 66

则称该排序方法是**不稳定**的

1. 插入类

将无序子序列中的一个或几个记录“**插入**”到有序序列中，从而增加记录的有序子序列的长度。



2. 交换类

通过“**交换**”无序序列中的记录从而得到其中关键字最小或最大的记录，并将它加入到有序子序列中，以此方法增加记录的有序子序列的长度。



3. 选择类

从记录的无序子序列中“选择”关键字最小或最大的记录，并将它加入到有序子序列中，以此方法增加记录的有序子序列的长度。



4. 归并类

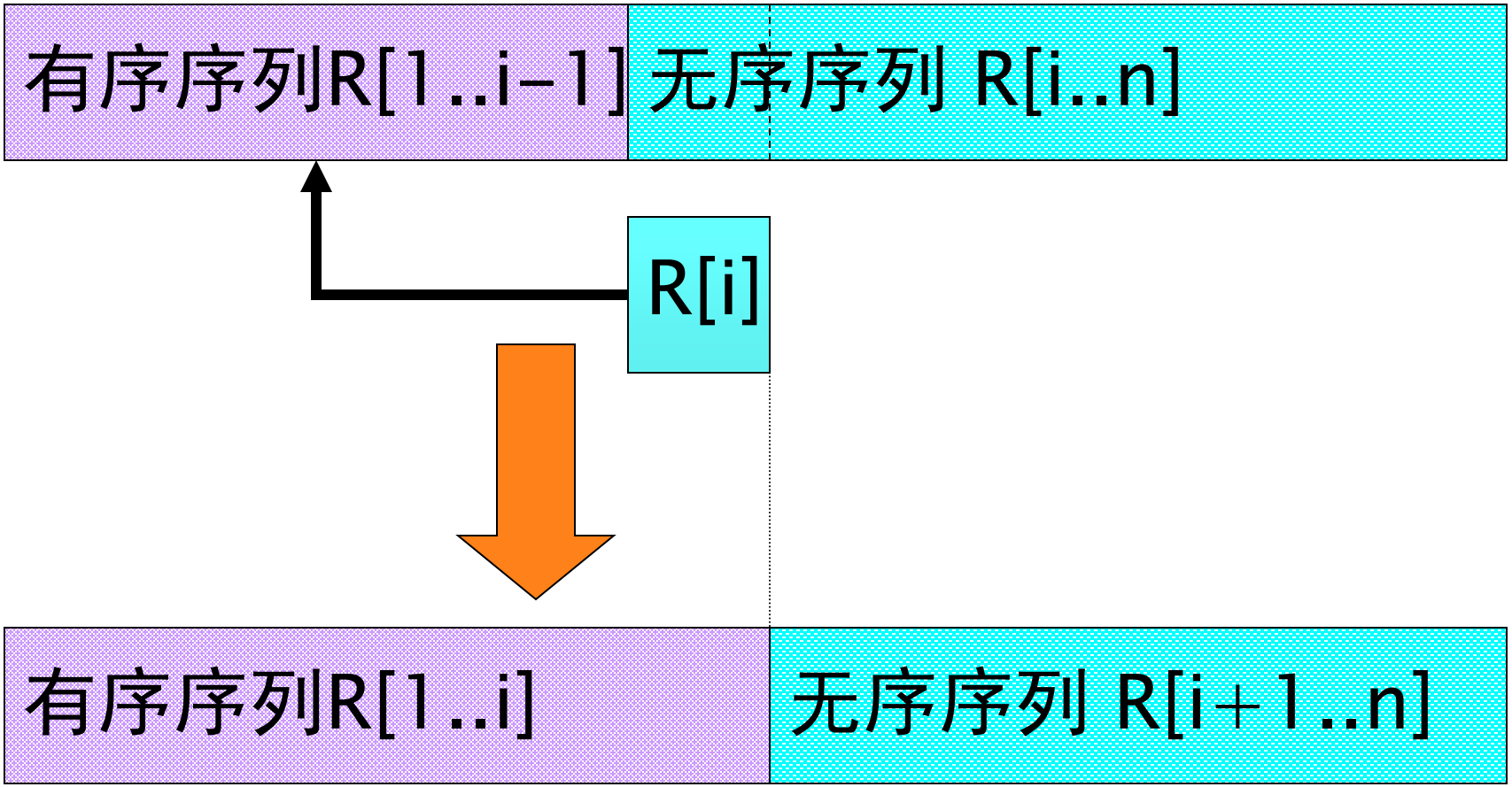
通过“归并”两个或两个以上的记录有序子序列，逐步增加记录有序序列的长度。

5. 其它方法

10.2

插入排序

一趟直接插入排序的基本思想：



实现“一趟插入排序”可分三步进行：

1．在R[1..i−1]中**查找**R[i]的插入位置，

R[1..j].key ≤ R[i].key < R[j+1..i−1].ke

2．将R[j+1..i−1]中的所有**记录**均**后移**一个位置；

3．将R[i] **插入**(复制)到R[j+1]的位置上。

不同的具体实现方法导致不同的算法描述

- 直接插入排序（基于顺序查找）
- 折半插入排序（基于折半查找）
- 表插入排序（基于链表存储）
- 希尔排序（基于逐趟缩小增量）

一、直接插入排序

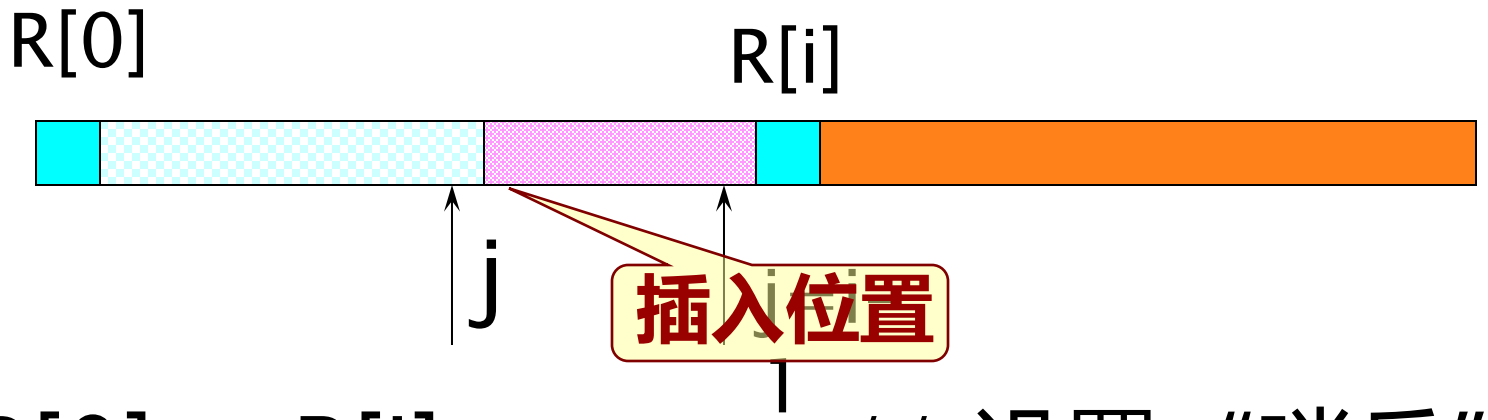
利用 “顺序查找”实现

“在R[1..i−1]中**查找**R[i]的插入位置”

算法的实现要点：

- 从R[i-1]起向前进行顺序查找，

  监视哨设置在R[0]；



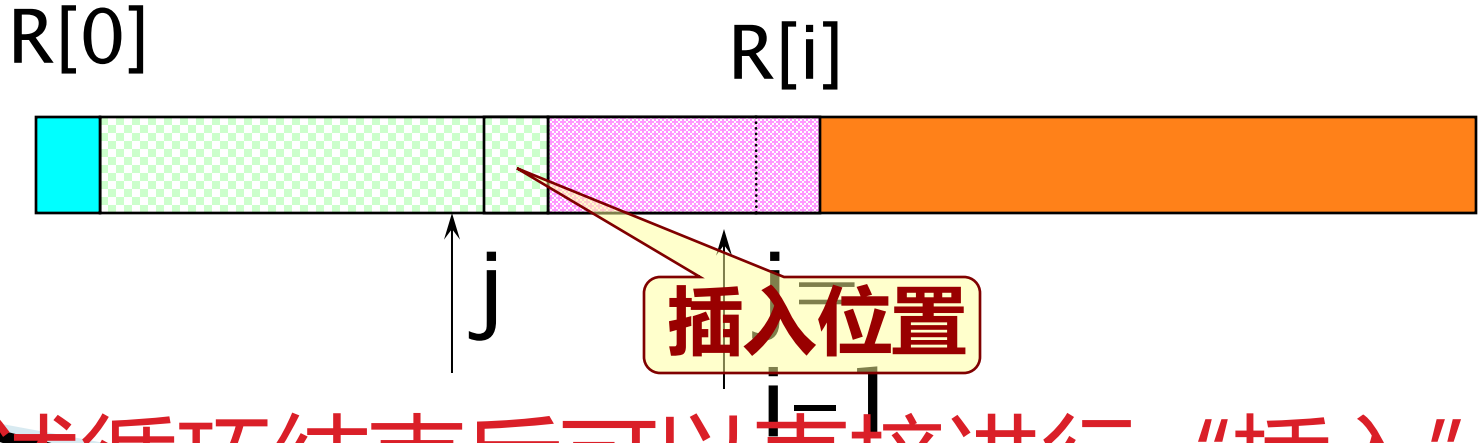

```
R[0] = R[i];                    // 设置“哨兵”
for (j=i-1; R[0].key<R[j].key; --j);
                                // 从后往前找
```

循环结束表明R[i]的插入位置为 *j +1*

- 对于在查找过程中找到的那些关键字不小于R[i].key的记录，并在查找的同时实现记录向后移动；

```
for (j=i-1; R[0].key<R[j].key; --j);
```

R[j+1] = R[j]



- 上述循环结束后可以直接进行“插入”

- 令 i = 2，3，…，n，

  实现整个序列的排序。

```
for ( i=2;  i<=n;  ++i )
    if (R[i].key<R[i-1].key)
      {  在 R[1..i-1]中查找R[i]的插入位置
          插入R[i] ;
      }
```

待排元素序列：[53]　27　36　15　69　42

第一次排序：　[27　53]　36　15　69　42

第二次排序：　[27　36　53]　15　69　42

第三次排序：　[15　27　36　53]　69　42

第四次排序：　[15　27　36　53　69]　42

第五次排序：　[15　27　36　42　53　69]

直接插入排序示例

对于有n个数据元素的待排序列，插入操作要进行n-1次

该算法适合于n 较小的情况，时间复杂度为$O(n^2)$

第九章 · 教材伪代码

直接插入排序

每趟开始时，前 i-1 项已有序。0 号单元暂存当前记录，作为监视哨。

```
void InsertionSort ( SqList &L ) {
  // 对顺序表 L 作直接插入排序。
   for ( i=2; i<=L.length; ++i )
       if (L.r[i].key < L.r[i-1].key) {
           L.r[0] = L.r[i];              // 复制为监视哨
           for ( j=i-1; L.r[0].key < L.r[j].key;  -- j )
               L.r[j+1] = L.r[j];         // 记录后移
           L.r[j+1] = L.r[0];         // 插入到正确位置
       }
} // InsertSort
```

把比当前记录大的元素从右向左依次后移，再把监视哨中的记录填入空位。

内部排序的**时间分析**：

实现内部排序的**基本操作**有两个：

（1）**“比较”**序列中两个关键字的大小；

（2）**“移动”**记录。

对于直接插入排序：

最好的情况（关键字在记录序列中顺序有序）：

“比较”的次数：

$$\sum_{i=2}^{n} 1 = n-1$$

“移动”的次数：

0

最坏的情况（关键字在记录序列中逆序有序）：

“比较”的次数：

$$\sum_{i=2}^{n} (i) = \frac{(n+2)(n-1)}{2}$$

“移动”的次数：

$$\sum_{i=2}^{n} (i+1) = \frac{(n+4)(n-1)}{2}$$

直接插入排序是一种稳定的排序方法

初始序列的为正序时时间复杂度为：O（n），初始序列的为逆序时时间复杂度为：O（n^2）

不是每趟排序后都将某个数据放置在它的最终位置

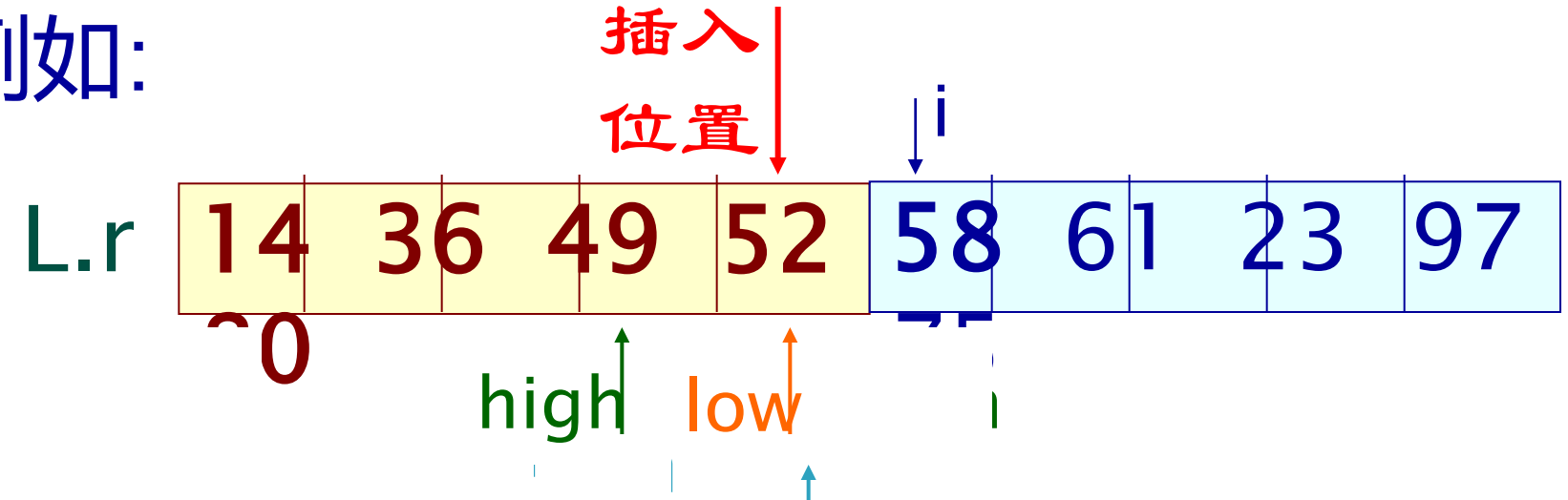
二、折半插入排序

因为 R[1..i-1] 是一个按关键字有序的有序序列，则可以利用**折半查找**实现“在R[1..i-1]中**查找**R[i]的插入位置”，如此实现的插入排序为**折半插入**排序。

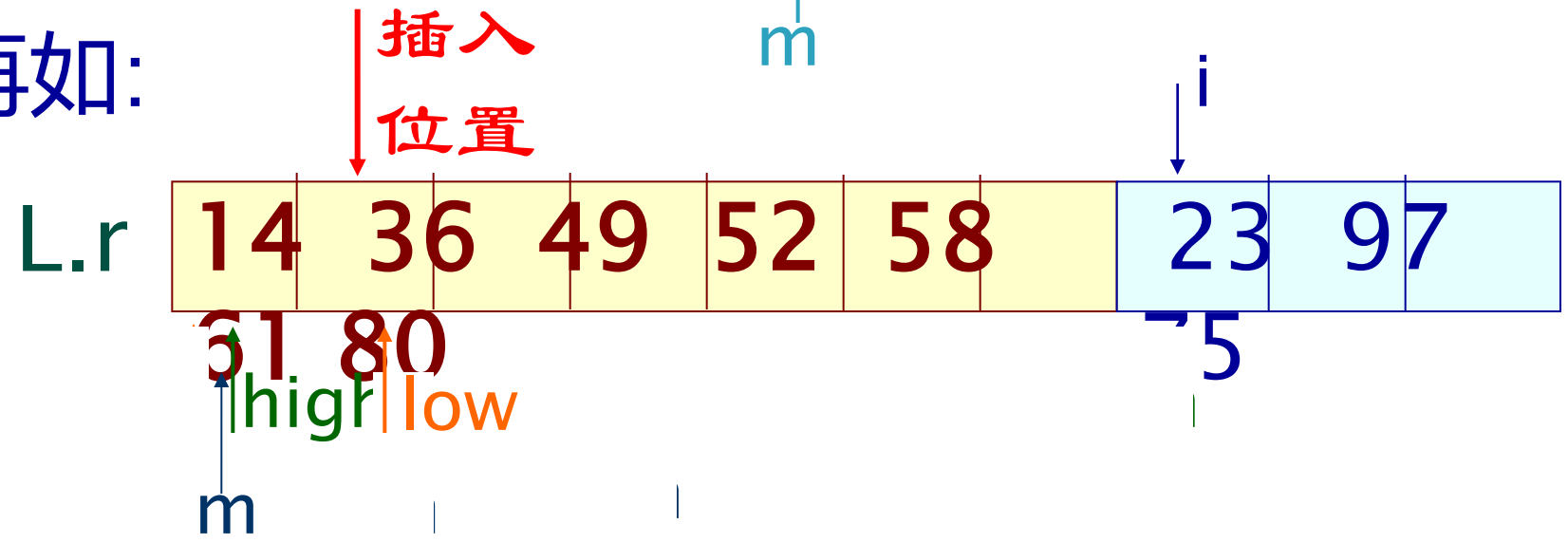
```
void BiInsertionSort ( SqList &L ) {
  for ( i=2; i<=L.length; ++i ) {
    L.r[0] = L.r[i];        // 将 L.r[i] 暂存到 L.r
    在 L.r[1..i-1]中折半查找插入位置;
    for ( j=i-1;  j>=high+1;  --j )
      L.r[j+1] = L.r[j];      // 记录后移
    L.r[high+1] = L.r[0];  // 插入
  } // for
} // BInsertSort
```

```
low = 1;   high = i-1;
while (low<=high) {
    m = (low+high)/2;              // 折半
    if (L.r[0].key < L.r[m].key)
            high = m-1;   // 插入点在低半
    else  low = m+1; // 插入点在高半区
}
```

例如:


插入
位置
i
L.r
14 36 49 52 58 61 23 97
high
low
m

再如:


插入
位置
i
L.r
14 36 49 52 58 23 97
high
low
m

三、表插入排序

为了减少在排序过程中进行的“移动”记录的操作，必须改变排序过程中采用的存储结构。利用静态链表进行排序，并在排序完成之后，一次性地调整各个记录相互之间的位置，即将每个记录都调整到它们所应该在的位置上。

```
void LInsertionSort (Elem SL[ ],  int n){
  // 对记录序列SL[1..n]作表插入排序
  SL[0].key = MAXINT ; // 0 号单元兼作链表头和哨兵
  SL[0].next = 1;  SL[1].next = 0;
  for ( i=2; i<=n; ++i )
     for ( j=0, k = SL[0].next;SL[k].key<
                SL[i].key ; j=k, k=SL[k].next
        { SL[j].next = i;  SL[i].next = k; }
       // 结点i插入在结点j和结点k之间
}// LinsertionSort
```

如何在排序之后调整记录序列?

算法中使用了三个指针:

其中: p指示第i个记录的当前位置

i指示第i个记录应在的位置

q指示第i+1个记录的当前位置

```
void Arrange ( Elem SL[ ], int n ) {
  p = SL[0].next;    // p指示第一个记录的当前位置
  for ( i=1; i<n; ++i ) {
    while (p<i)  p = SL[p].next;
    q = SL[p].next;      // q指示尚未调整的表
    if ( p!= i ) {
      SL[p]←→SL[i]; // 交换记录，使第i个记录到位
      SL[i].next = p;    // 指向被移走的记录
    }
    p = q;          // p指示尚未调整的表尾，
                    // 为找第i+1个记录作准备
  }
} // Arrange
```

四、希尔排序（又称缩小增量排序）

基本思想：对待排记录序列先作“宏观”调整，再作“微观”调整。

所谓“宏观”调整，指的是，“跳跃的插入排序。

具体做法为：

将记录序列分成若干子序列，分别对每个子序列进行插入排序。

例如：将 n 个记录分成 d 个子序列：

{ R[1]，R[1+d]，R[1+2d]，…，R[1+kd] }

{ R[2]，R[2+d]，R[2+2d]，…，R[2+kd] }

…

{ R[d]，R[2d]，R[3d]，…，R[kd]，R[(k+1)d] }

其中，d 称为增量，它的值在排序过程中从大到小逐渐缩小，直至最后一趟排序减为 1。

例如：

16	25	12	30	47	11	23	36	9	18

第一趟希尔排序，设增量 d =5

11	23	12	9	18	16	25	36	30	47

第二趟希尔排序，设增量 d = 3

9	18	12	11	23	16	25	31	30	47	3

第三趟希尔排序，设增量 d = 1

9	11	12	16	18	23	25	30	31	36

```
void ShellInsert ( SqList &L, int dk ) {
  for ( i=dk+1; i<=n; ++i ) // 在间隔为 dk 的子序列内插入
    if ( L.r[i].key< L.r[i-dk].key) {
      L.r[0] = L.r[i];                    // 暂存在R[0]
      for (j=i-dk;  j>0&&(L.r[0].key<L.r[j]
            j-=dk)
        L.r[j+dk] = L.r[j];   // 记录后移，查找插
      L.r[j+dk] = L.r[0];                 // 插入
    } // if
} // ShellInsert
```

```
void ShellSort (SqList &L, int dlta[], int
{    // 增量为dlta[]的希尔排序
     for (k=0; k<t; ++t) // 循环步进应为 ++k
          ShellInsert(L, dlta[k]);
               //一趟增量为dlta[k]的插入排序
} // ShellSort
```

*Gap(间隔)*的取法有多种。最初shell 提出取 $gap = \lfloor n/2 \rfloor$，$gap = \lfloor gap/2 \rfloor$，直到$gap = 1$。后来knuth 提出取$gap = \lfloor gap/3 \rfloor + 1$。还有人提出都取奇数为好，也有人提出各$gap$互质为好

Knuth利用大量的实验统计资料得出，当 n 很大时，关键码平均比较次数和对象平均移动次数大约在 $n^{1.25}$ 到 $1.6n^{1.25}$ 的范围内。这是在利用直接插入排序作为子序列排序方法的情况下得到的

希尔排序是一种不稳定的排序方法

初始序列的不同会影响算法的效率

不是每趟排序后都将某个数据放置在它的最终位置

10.3 交换排序

一、起泡排序

二、一趟快速排序

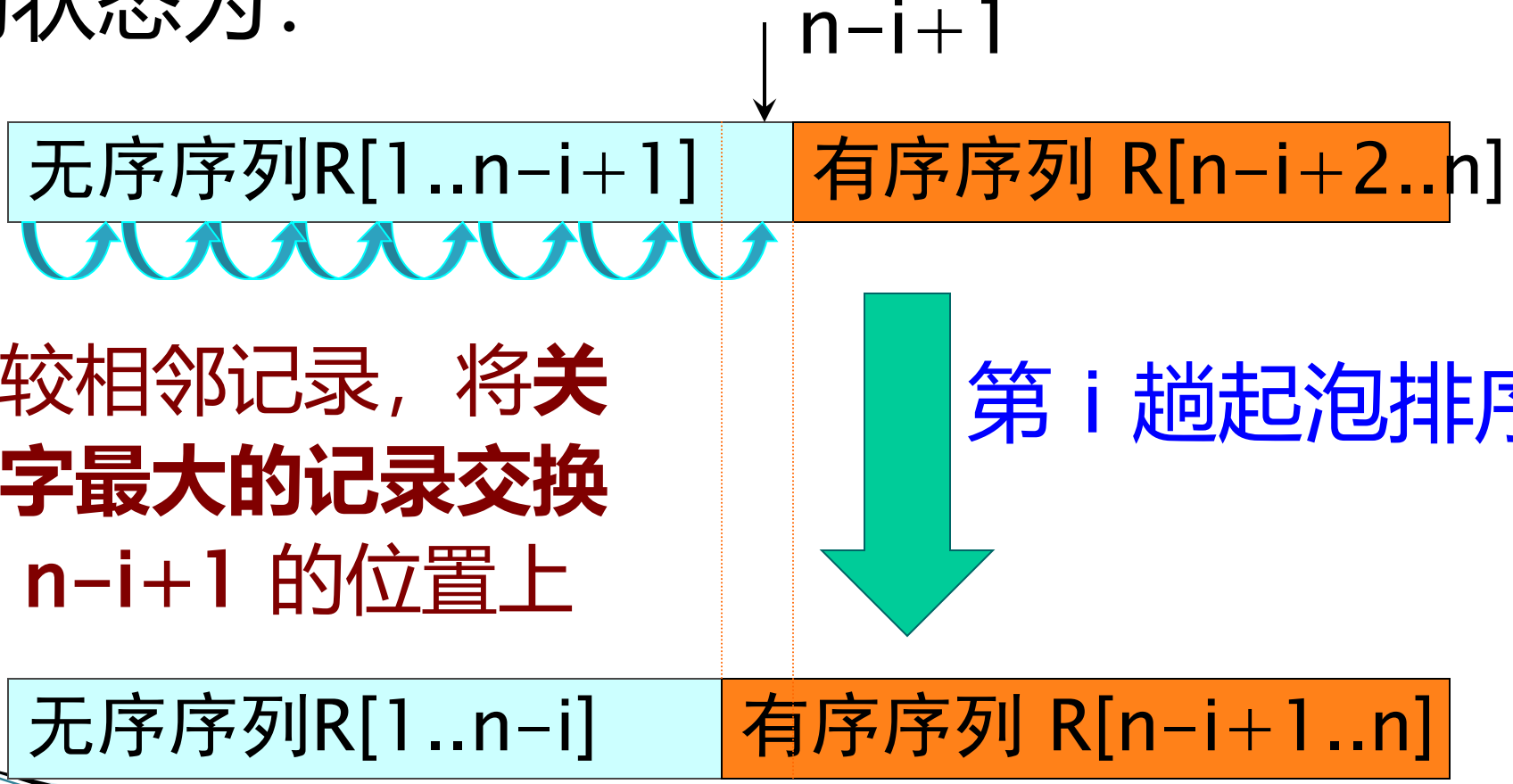
三、快速排序

四、快速排序的时间分析



一、起泡排序

假设在排序过程中，记录序列R[1..n]的状态为：



```
void BubbleSort(Elem R[ ], int n) {
    i = n;
    while (i >1) {
        lastExchangeIndex = 1;
        for (j = 1;  j < i;  j++)
            if (R[j+1].key < R[j].key) {
                Swap(R[j], R[j+1]); // 仅交换严格逆序的相邻记录
                lastExchangeIndex = j;   //记下进行交换的记录位
            } //if
        i = lastExchangeIndex; // 本趟进行过交换的
    } // while                          // 最后一个记录的
} // BubbleSort
```



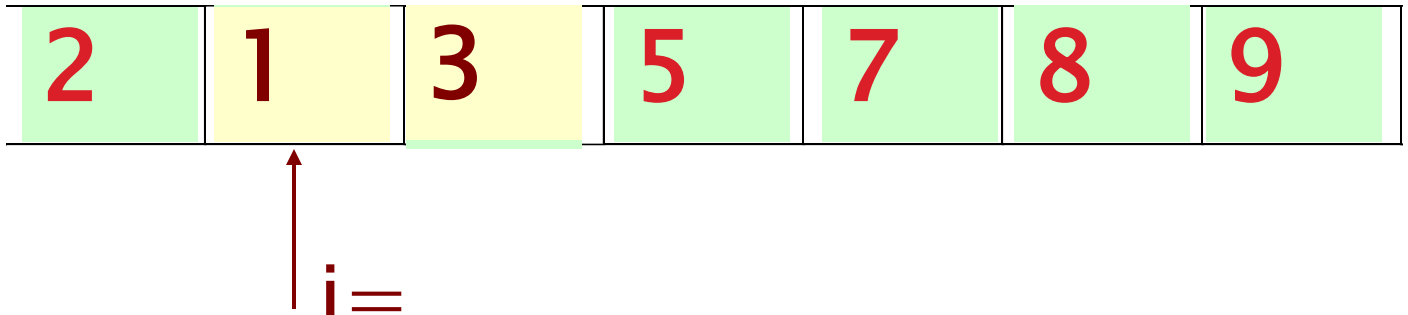
注意:

1. 起泡排序的结束条件为，

最后一趟没有进行“交换记录”。

2. 一般情况下，每经过一趟“起泡”，“i 减一”，但并不是每趟都如此。

例如:




```
for (j = 1; j < i; j++)  if (R[j+1].key < R[j].k
```

时间分析：

最好的情况（关键字在记录序列中顺序有序）：

只需进行一趟起泡

“比较”的次数：

n-1

“移动”的次数：

0

最坏的情况（关键字在记录序列中逆序有序）：

需进行n-1趟起泡

“比较”的次数：

$$\sum_{i=n}^{2}(i-1)=\frac{n(n-1)}{2}$$

“移动”的次数：

$$3\sum_{i=n}^{2}(i-1)=\frac{3n(n-1)}{2}$$

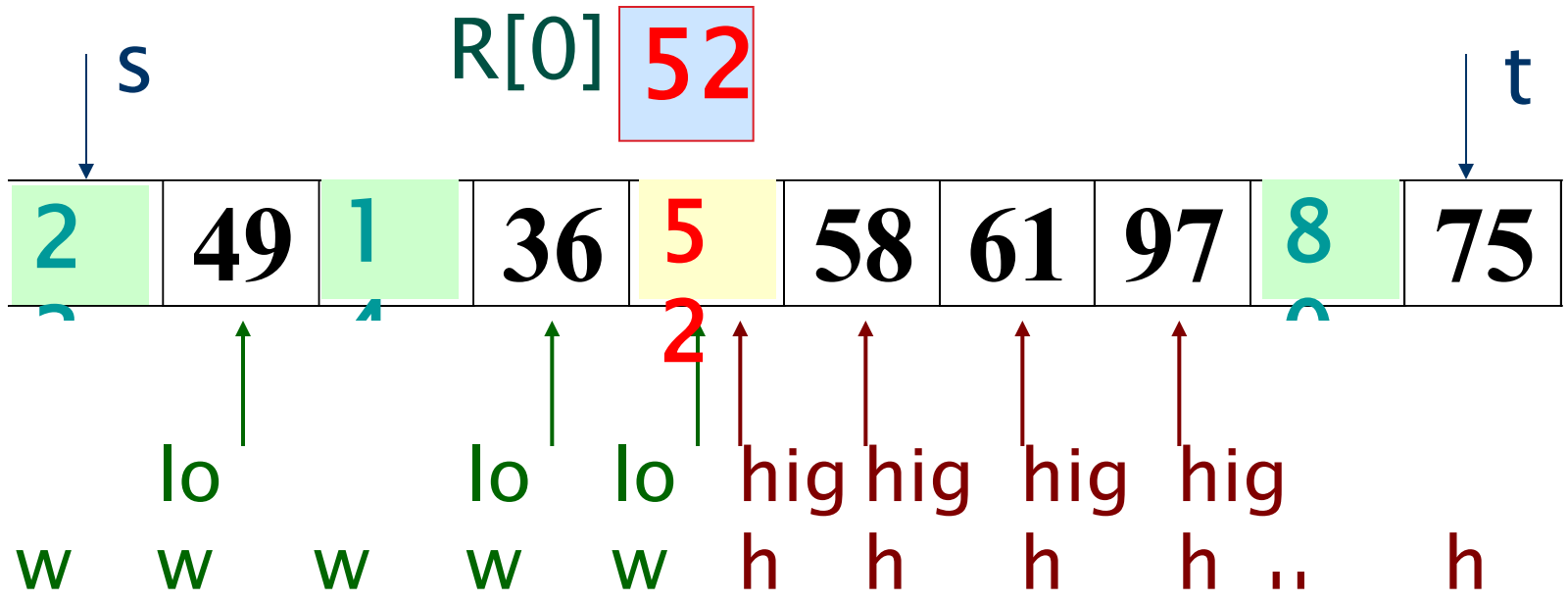
二、一趟快速排序（一次划分）

目标：找一个记录，以它的关键字作为**“枢轴”**，凡其**关键字小于枢轴**的记录均**移动至该记录之前**，反之，凡**关键字大于枢轴**的记录均**移动至该记录之后。**

致使**一趟排序**之后，记录的无序序列R[s..t]将**分割成两部分**：R[s..i−1]和R[i+1..t]，且

$$R[j].key \le R[i].key \le R[j].key$$

例如



设 R[s]=52 为枢轴

将 R[high].key 和 枢轴的关键字进行比较，要求R[high].key ≥ 枢轴的关键字

将 R[low].key 和 枢轴的关键字进行比较，要求R[low].key ≤ 枢轴的关键字

可见，经过“**一次划分**”，将关键字序列

52, 49, 80, 36, 14, 58, 61, 97

调整为: 23, 49, 14, 36, (52) 58, 61, 97,

在调整过程中，设立了两个指针： low和high，它们的初值分别为： s 和 t，

之后逐渐减小 high，增加 low，并保证

R[high].key≥52，和

R[low].key≤52，否则进行记录的“交换”。



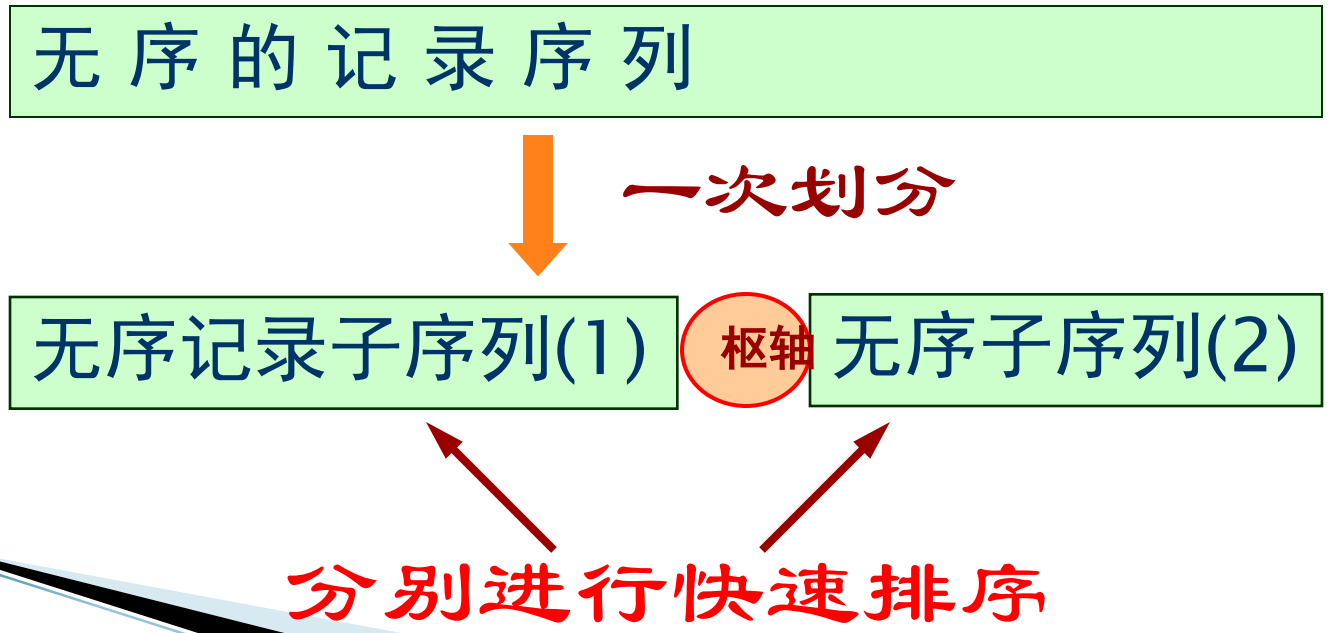
```
int Partition (RedType& R[], int low, int high) {
  pivotkey = R[low].key; // 取区间首记录为枢轴
  while (low<high) {
    while (low<high && R[high].key>=pivotkey
      --high;
    R[low]←→R[high];
    while (low<high && R[low].key<=pivotkey)
      ++low;
    R[low]←→R[high];
  }
  return low;              // 返回枢轴所在位置
} // Partition
```

```
int Partition (RedType R[], int low, int high) {
    R[0] = R[low];  pivotkey = R[low].key;  // 枢
   while (low<high) {
      while(low<high&& R[high].key>=pivotke
            -- high;       // 从右向左搜索
      R[low] = R[high];
      while (low<high && R[low].key<=pivotke
            ++ low;       // 从左向右搜索
      R[high] = R[low];
   }
   R[low] = R[0];      return low;
}// Partition
```



三、快速排序

首先对无序的记录序列进行“**一次划分**”，之后**分别**对分割所得两个子序列“**递归**”**进行快速排序**。



```
void QSort (RedType & R[], int s, int t )
  // 对记录序列R[s..t]进行快速排序
  if (s < t-1) {                    // 长度大于1
    pivotloc = Partition(R, s, t);
                                    // 对 R[s..t] 进行一次划分
    QSort(R, s, pivotloc-1);
          // 对低子序列递归排序，pivotloc是枢轴位
    QSort(R, pivotloc+1, t); // 对高子序列递归
  }
} // QSort
```

第一次调用函数 Qsort 时，待排序记录序列的上、下界分别为 1 和 L.length。

```
void QuickSort( SqList & L) {
    // 对顺序表进行快速排序
    QSort(L.r, 1, L.length);
} // QuickSort
```



四、快速排序的时间分析

假设一次划分所得枢轴位置 $i=k$，则对 n 个记录进行快排所需时间：

$$T(n) = T_{pass}(n) + T(k-1) + T(n-k)$$

其中 $T_{pass}(n)$为对 n 个记录进行一次划分所需时间。

若待排序列中记录的关键字是随机分布的，则 k 取 1 至 n 中任意一值的可能性相同。

由此可得快速排序所需时间的平均值为：

$$T_{avg}(n) = Cn + \frac{1}{n}\sum_{k=1}^{n}\left[T_{avg}(k-1) + T_{avg}(n-k)\right]$$

设 $T_{avg}(1) \le b$

则可得结果：

$$T_{avg}(n) < (\frac{b}{2} + 2c)(n+1)\ln(n+1)$$

结论：快速排序的时间复杂度为O(nlogn)

若待排记录的初始状态为按关键字有序时，快速排序将蜕化为起泡排序，其时间复杂度为$O(n^2)$。

为避免出现这种情况，需在进行一次划分之前，进行“**予处理**”，即：

先对 R(s).key，R(t).key 和 R[⌊(s+t)/2⌋].key，进行相互比较，然后**取**关键字为 “**三者之中**”的记录**为枢轴**记录。

快速排序是一种不稳定的排序方法

初始序列的不同会影响算法的效率

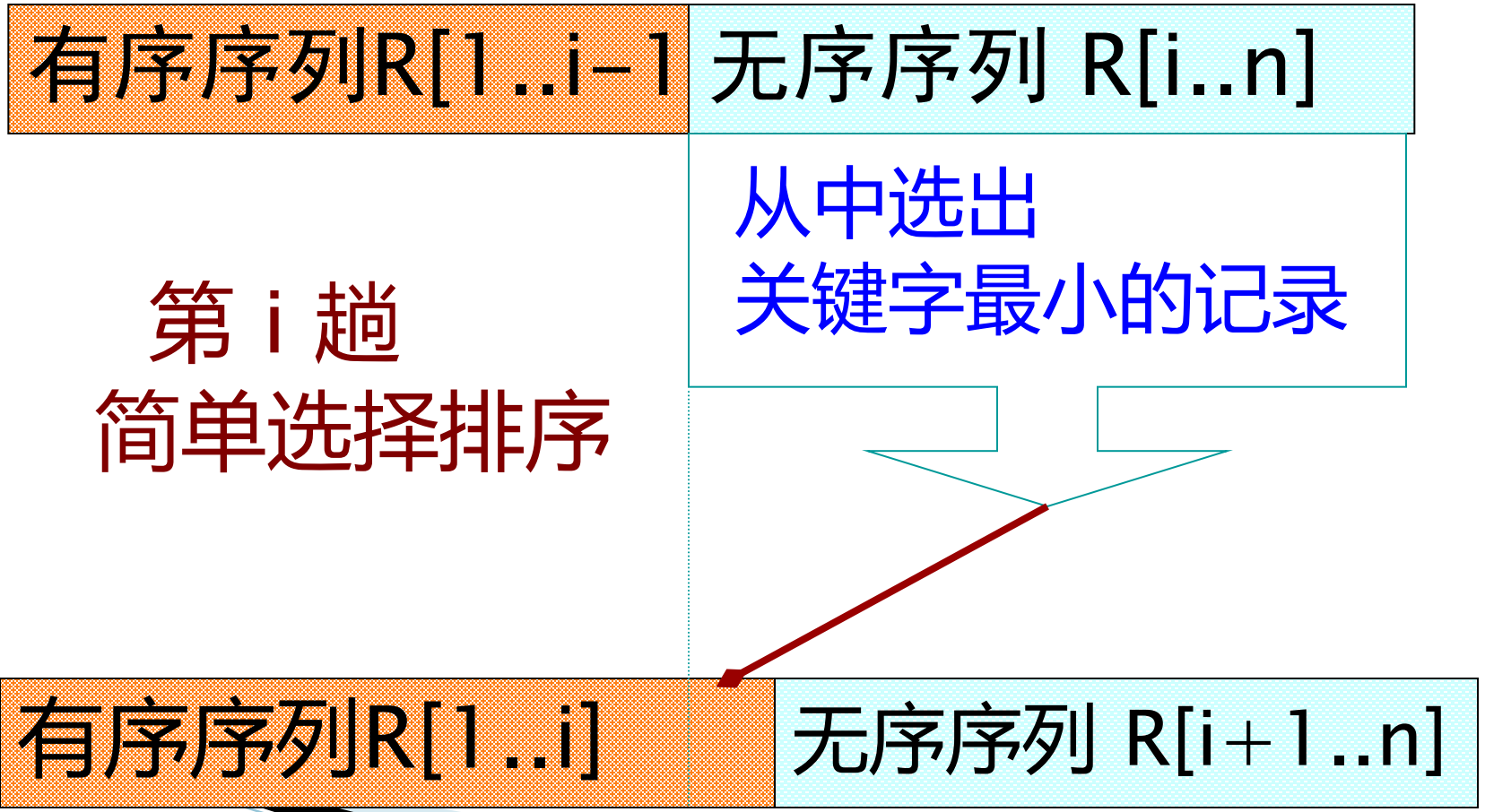
每趟排序后都将某个数据放置在它的最终位置

10.4 堆排序

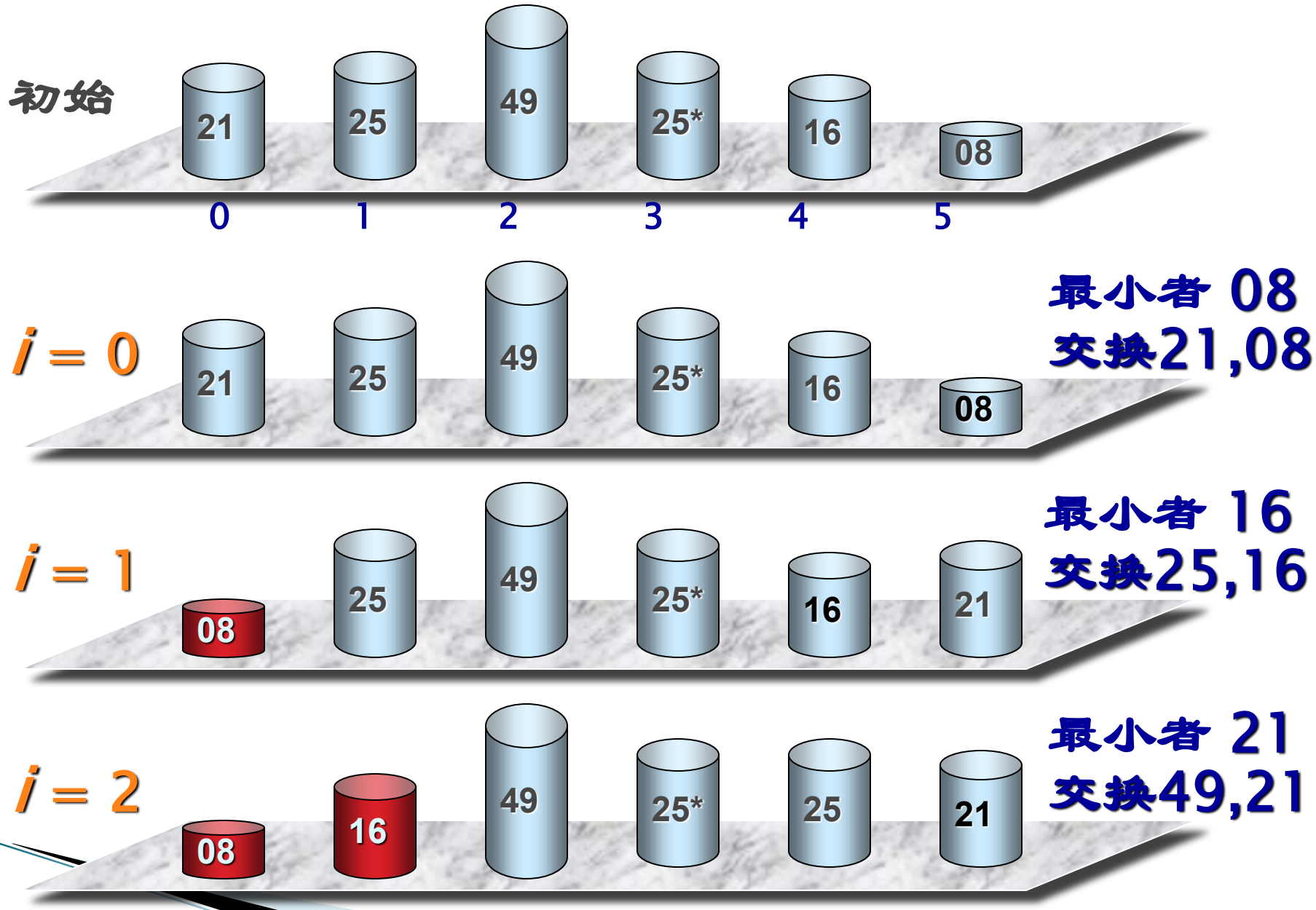
- 简单选择排序
- 堆排序

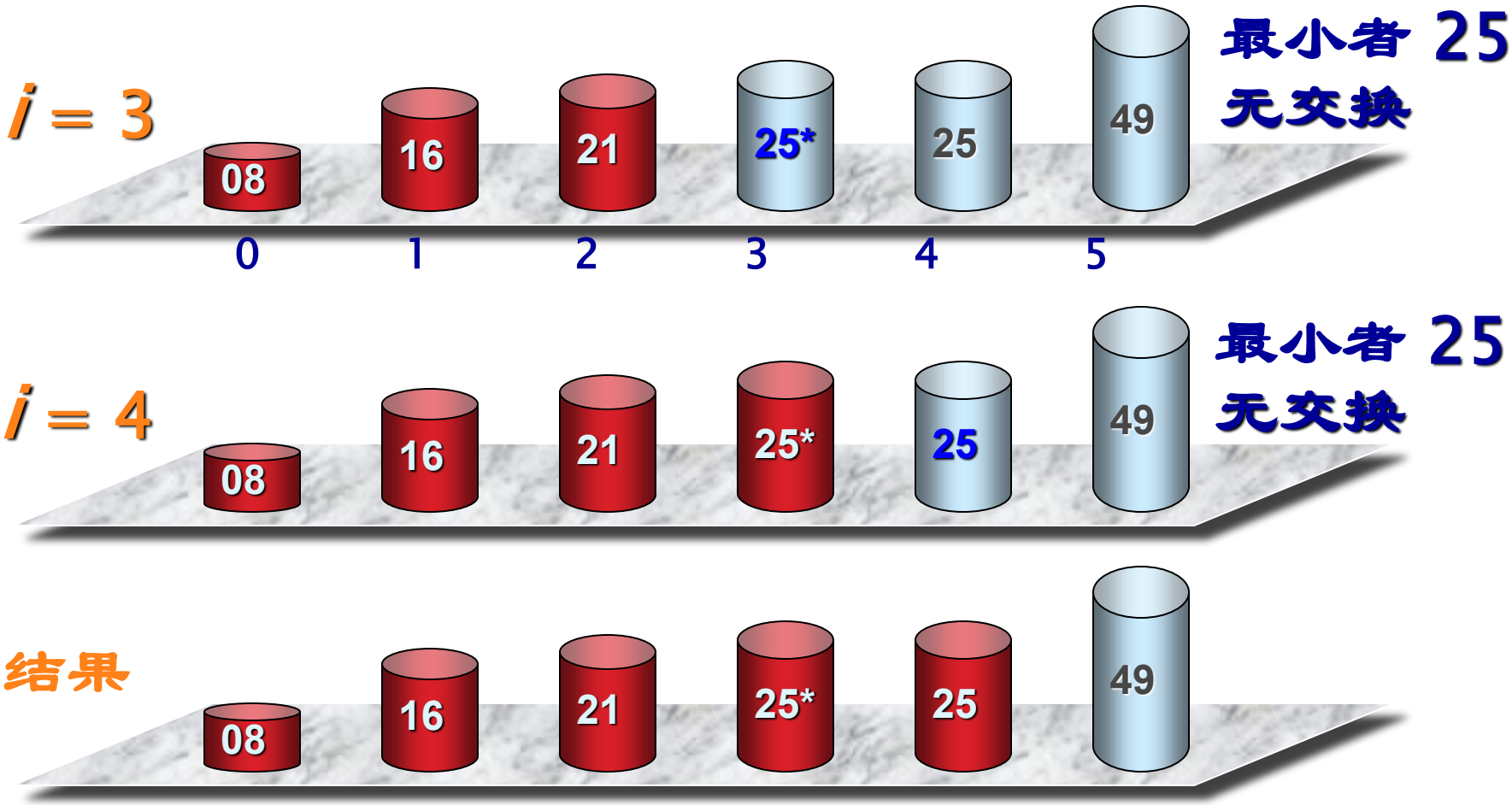
一、简单选择排序

假设排序过程中，待排记录序列的状态为：




k k k
i=1 初始： [13 38 65 97 49 13 27]
j j j j j j
k k
i=2 一趟： 13 27 38 65 97 76 49 27]
j j j j j
二趟： 13 27 [65 97 76 49 38]
三趟： 13 27 38 [97 76 49 65]
四趟： 13 27 38 49 [76 97 65]
五趟： 13 27 38 49 65 [97 76]
六趟： 13 27 38 49 65 76 [97]
排序结束： 13 27 38 49 65 76 97


初始
21
25
49
25*
16
08
0
1
2
3
4
5
i = 0
最小者 08
交换21,08
i = 1
最小者 16
交换25,16
i = 2
最小者 21
交换49,21


i = 3
08
16
21
25*
25
49
0
1
2
3
4
5
最小者 25*
无交换
i = 4
08
16
21
25*
25
49
最小者 25
无交换
结果
08
16
21
25*
25
49

各趟排序后的结果

简单选择排序的算法描述如下：

```
void SelectSort (Elem R[], int n ) {
  // 对记录序列R[1..n]作简单选择排序。
  for (i=1; i<n; ++i) {
                  // 选择第 i 小的记录，并交换到位
    j = SelectMinKey(R, i);
                  // 在 R[i..n] 中选择关键字最小的记录
    if (i!=j)  R[i]←→R[j];
                        // 与第 i 个记录交换
  }
} // SelectSort
```

时间性能分析

对 n 个记录进行简单选择排序，所需进行的 **关键字间的比较次数** 总计为：

$$\sum_{i=1}^{n-1}(n-i)=\frac{n(n-1)}{2}$$

移动记录的次数，最小值为 0，最大值为3(n−1)。

简单选择排序是一种不稳定的排序方法

初始序列的不同不会影响算法的效率

每趟排序后都将某个数据放置在它的最终位置

二、堆排序

堆的定义:

堆是满足下列性质的数列{r_1, r_2, …, r_n}:

$$\begin{cases} r_i \le r_{2i} \\ r_i \le r_{2i+1} \end{cases}$$ (小顶堆) 或 $$\begin{cases} r_i \ge r_{2i} \\ r_i \ge r_{2i+1} \end{cases}$$ (大顶堆)

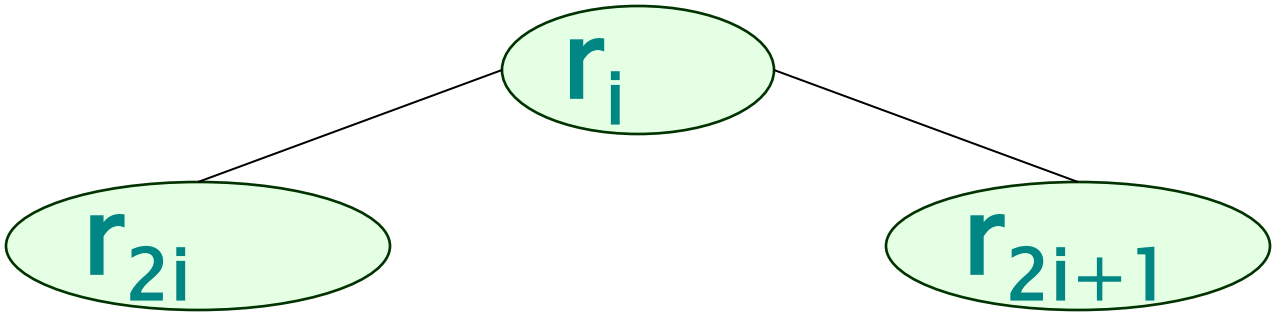
例如:

{12, 36, 27, 65, 40, 34, 98, 81, 73, 50} 是小顶堆

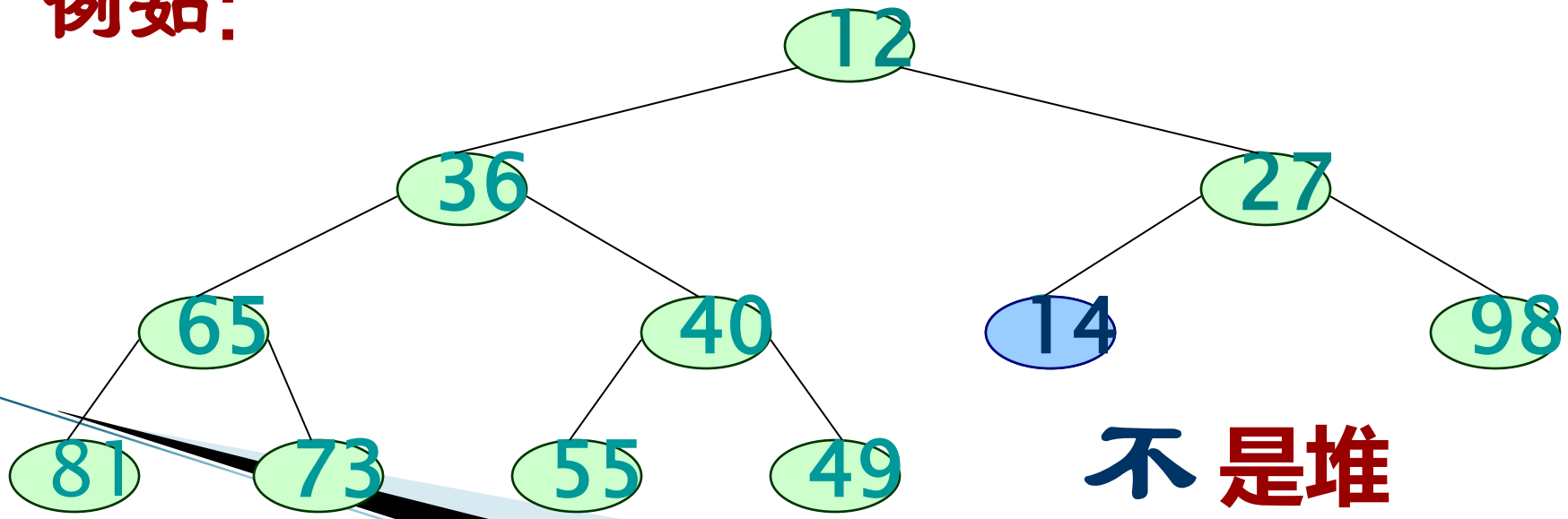
{12, 36, 27, 65, 40, 14, 98, 81, 73, 50} 不是堆

若将该数列视作完全二叉树，

则 r_{2i} 是 r_i 的左孩子； r_{2i+1} 是 r_i 的右孩子。



例如：

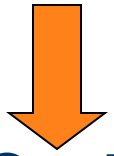


不是堆

堆排序即是利用**堆的特性**对记录序列进行排序的一种排序方法。

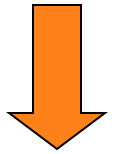
例如：

{ 40, 55, 49, 73, 12, 27, 98, 81, 64,



建大顶堆

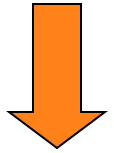
{ 98, 81, 49, 73, 36, 27, 40, 55, 64,



交换 98 和 12

{ 12, 81, 49, 73, 36, 27, 40, 55, 64,

经过筛选



重新调整为大顶堆

{ 81, 73, 49, 64, 36, 27, 40, 55,

12, 98 }

定义堆类型为:

```
typedef  SqList HeapType;
                                    // 堆采用顺序表表示
```

两个问题:

如何“建堆”？

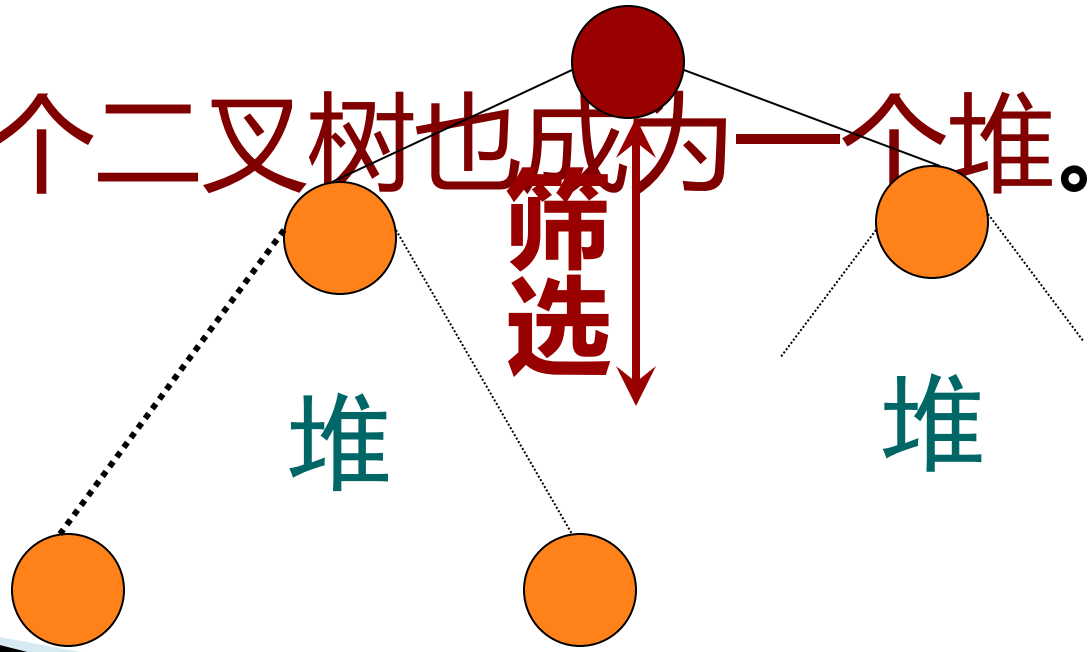
如何“筛选”？



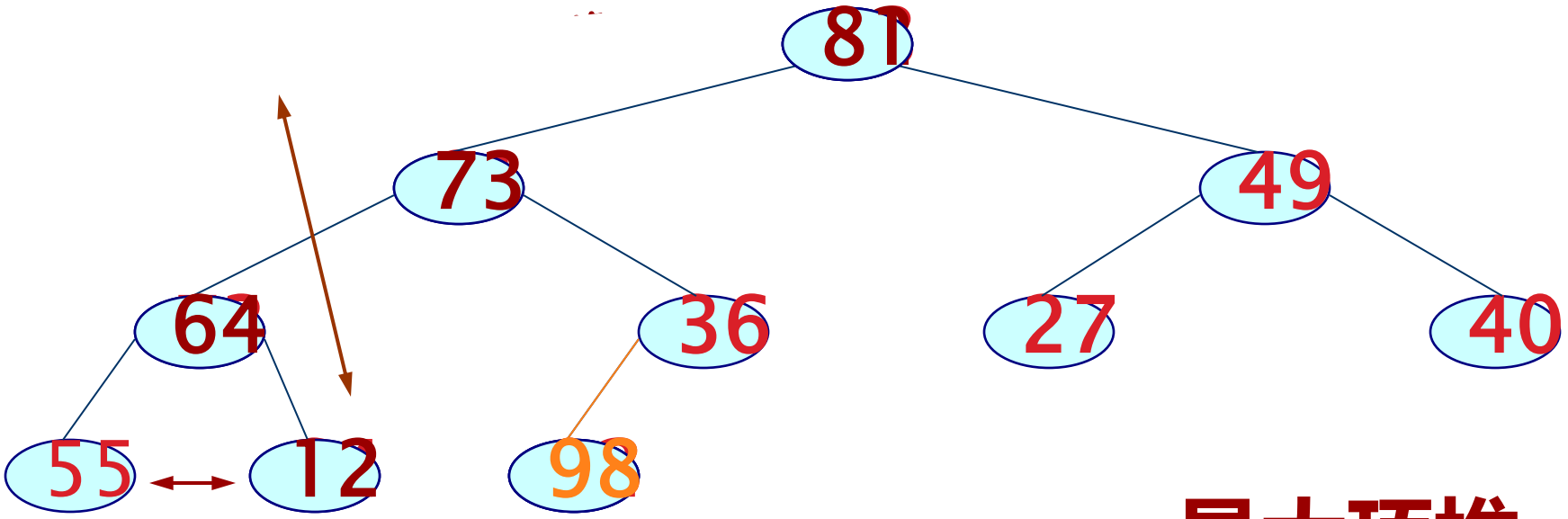
所谓“**筛选**”指的是，对一棵左/右子树

均为堆的完全二叉树，“**调整**”**根结点**

使整个二叉树也成为一个堆。



例如：



是大顶堆

但在 98 和 12 进行互换之后，它就不是堆了

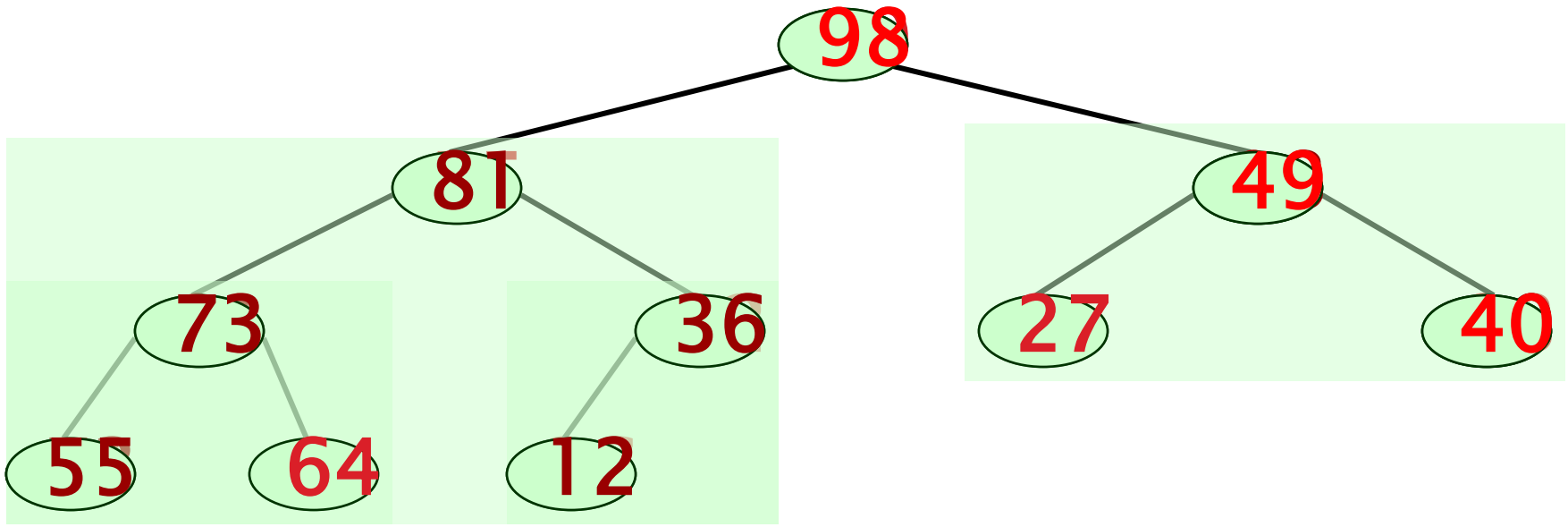
因此，需要对它进行“筛选”。

```
void HeapAdjust (RcdType &R[], int s, int m)
{   // 已知 R[s..m]中记录的关键字除 R[s] 之外均
    // 满足堆的特征，本函数自上而下调整 R[s] 的
    rc = R[s];
    for ( j=2*s; j<=m; j*=2 ) { // j 初值指向
        // 关键字，使 R[s..m] 也成为一个大顶堆
        自上而下的筛选过程;  // 沿较大孩子所在的分支下沉
    }
    R[s] = rc;  // 将调整前的堆顶记录插入到 s 位
```

```
if ( j<m && R[j].key<R[j+1].key ) ++j;
                // 左/右“子树根”之间先进行相互比较
                // 令 j 指示关键字较大记录的位置
if ( rc.key >= R[j].key ) break;
            // 再作“根”和“子树根”之间的比较，
                // 若“>=”成立，则说明已找到 rc 的插
                // 入位置 s，不需要继续往下调整
R[s] = R[j]; s = j;
            // 否则记录上移，尚需继续往下调整
```

建堆是一个从下往上进行“筛选”的过程。

例如：排序之前的关键字序列为



现在，左/右子树都已经调整为堆，最后只要调整根结点，使整个二叉树是个“堆”即可。

```
void HeapSort ( HeapType &H ) {
  // 对顺序表 H 进行堆排序
  for ( i=H.length/2;   i>0;   --i )
      HeapAdjust ( H.r, i, H.length );     // 建大
  for ( i=H.length; i>1; --i ) {
      H.r[1]←→H.r[i]; // 最大记录移到有序区的前端
          // 将堆顶记录和当前未经排序子序列
          //  H.r[1..i]中最后一个记录相互交换
      HeapAdjust(H.r, 1, i-1);  // 对 H.r[1] 进
  }
} // HeapSort
```



堆排序的时间复杂度分析：

1. 对深度为 k 的堆，“筛选”所需进行的关键字比较的次数至多为2(k-1)；

2. 对 n 个关键字，建成深度为$h(=\lfloor \log_2 n \rfloor+1)$的堆，所需进行的关键字比较的次数至多 $4n$；

3. 调整“堆顶” $n-1$ 次，总共进行的关键字比较的次数不超过

$$2(\lfloor \log_2(n-1) \rfloor+\lfloor \log_2(n-2) \rfloor+\ldots+\log_2 2) < 2n$$

因此，堆排序的时间复杂度为$O(n\log n)$。

堆排序是一种不稳定的排序方法

初始序列的不同会影响算法的效率

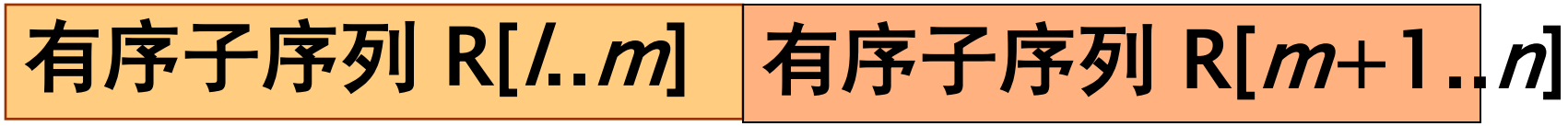
每趟排序后都将某个数据放置在它的最终位置

10.5 归并排序

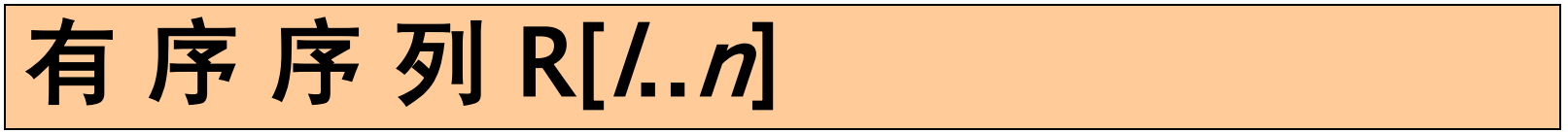
归并排序的过程基于下列**基本思想**进行：

将两个或两个以上的有序子序列 “归并” 为一个有序序列。

在内部排序中，通常采用的是2-路归并排序。即：将两个**位置相邻**的记录有序子序列



归并为**一个**记录的有序序列。



这个操作对顺序表而言，是轻而易举的。

```
void Merge (RcdType SR[], RcdType &TR[],
                                        int i, int m
  // 将有序的记录序列 SR[i..m] 和 SR[m+1..n]
  // 归并为有序的记录序列 TR[i..n]
 for (j=m+1, k=i;  i<=m && j<=n;  ++k)
  {                        // 将SR中记录由小到大地并入TR
    if (SR[i].key<=SR[j].key)  TR[k] = SR[i++];
    else TR[k] = SR[j++]; // 仅在左段关键字更大时取右段
  }
      ...    ...
} // Merge
```



```
if (i<=m) TR[k..n] = SR[i..m];
                    // 将剩余的 SR[i..m] 复制到 T

if (j<=n) TR[k..n] = SR[j..n];
                    // 将剩余的 SR[j..n] 复制到 T
```



归并排序的算法

如果记录无序序列 R[s..t] 的两部分

R[$s..\lfloor(s+t)/2\rfloor$] 和

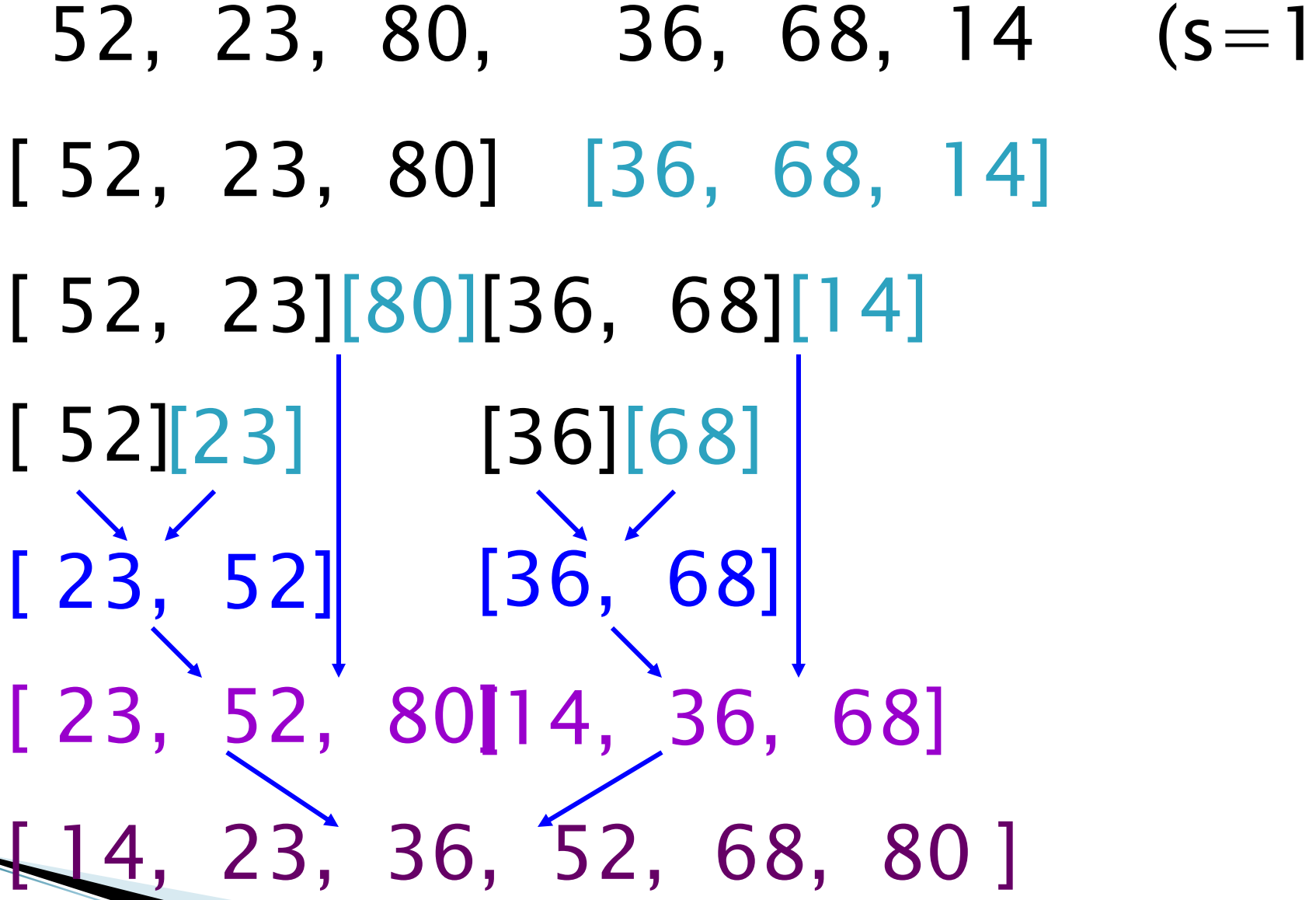
R[$\lfloor(s+t)/2\rfloor+1..t$]

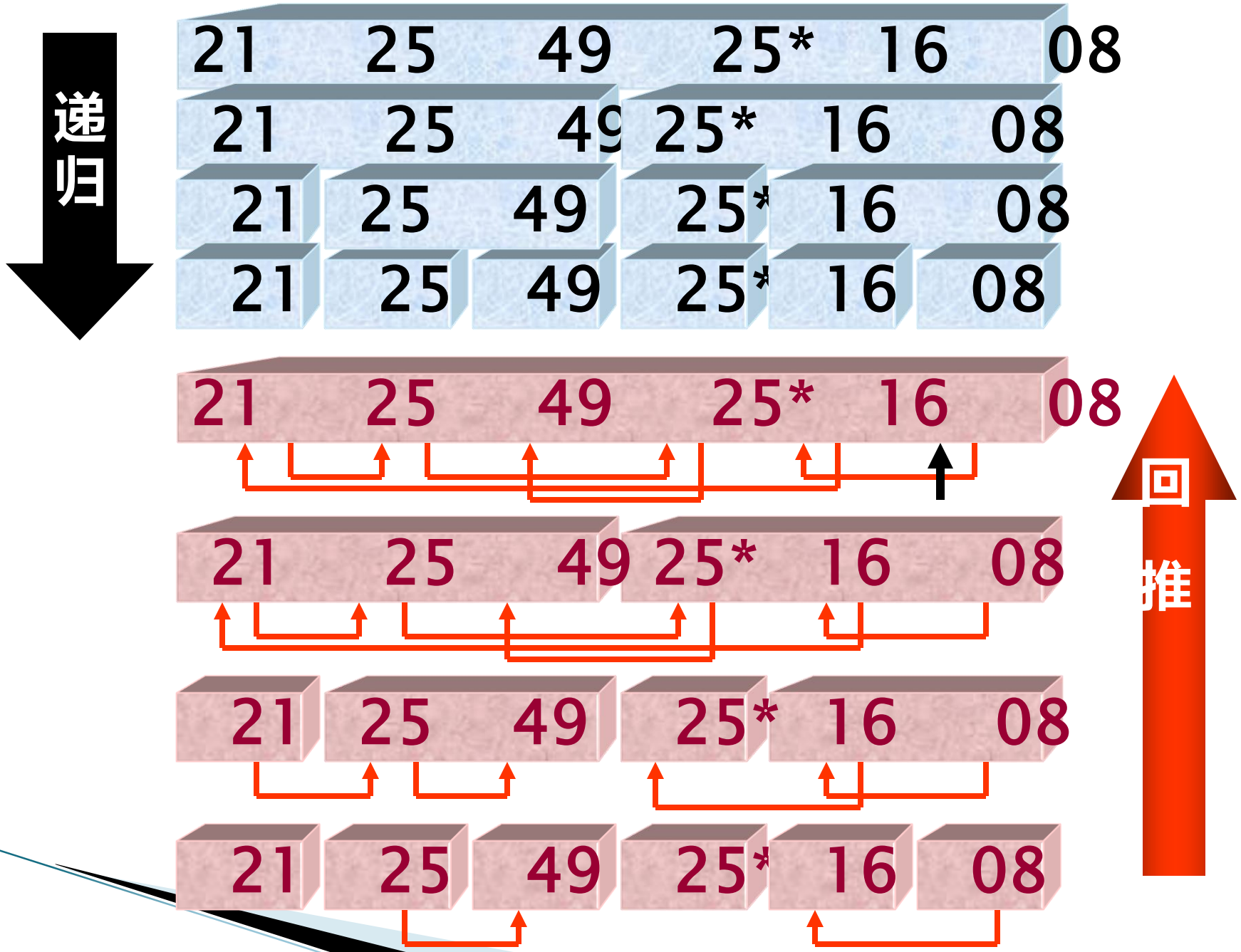
分别按关键字有序，

则利用上述归并算法很容易将它们归并成整个记录序列是一个有序序列。

由此，应该先分别对这两部分进行2-路归并排序。

例如：


52, 23, 80, 36, 68, 14 (s=1
[52, 23, 80] [36, 68, 14]
[52, 23][80][36, 68][14]
[52][23] [36][68]
[23, 52] [36, 68]
[23, 52, 80] [14, 36, 68]
[14, 23, 36, 52, 68, 80]


递归
21 25 49 25* 16 08
21 25 49 25* 16 08
21 25 49 25* 16 08
21 25 49 25* 16 08
21 25 49 25* 16 08
21 25 49 25* 16 08
21 25 49 25* 16 08
21 25 49 25* 16 08
回推

```
void Msort ( RcdType SR[],
                        RcdType &TR1[], int s, int
  // 将SR[s..t] 归并排序为 TR1[s..t]
  if (s== =t) TR1[s]=SR[s]; // 单元素序列直接复制
  else
    {          …    …
     }
} // Msort
```



```
m = (s+t)/2;
        // 将SR[s..t]平分为SR[s..m]和SR[m+
Msort (SR, TR2, s, m);
    // 递归地将SR[s..m]归并为有序的TR2[s..m
Msort (SR, TR2, m+1, t);
    //递归地SR[m+1..t]归并为有序的TR2[m+1
Merge (TR2, TR1, s, m, t);
    // 将TR2[s..m]和TR2[m+1..t]归并到TR1
```



```
void MergeSort (SqList &L) {
  // 对顺序表 L 作2-路归并排序
  MSort(L.r, L.r, 1, L.length);
} // MergeSort
```

容易看出，对 n 个记录进行归并排序的时间复杂度为$O(n\log n)$。即：

每一趟归并的时间复杂度为 $O(n)$，

总共需进行 $\lceil \log_2 n \rceil$ 趟。



归并排序是一种稳定的排序方法

初始序列的不同不会影响算法的效率

不是每趟排序后都将某个数据放置在它的最终位置

10.6 基数排序

基数排序是一种借助“多关键字排序”的思想来实现“单关键字排序”的内部排序算法。

- 多关键字的排序
- 链式基数排序

一、多关键字的排序

***n* 个记录的序列** $\{R_1, R_2, \ldots, R_n\}$

对关键字 $(K_i^0, K_i^1, \ldots, K_i^{d-1})$ **有序**是指：

对于序列中任意两个记录 R_i 和 R_j $(1 \le i < j \le n)$ 都**满足**下列**(词典)有序**关系：

$$(K_i^0, K_i^1, \ldots, K_i^{d-1}) < (K_j^0, K_j^1, \ldots, K_j^{d-1})$$

其中： K^0 **被称为 “最主”位关键字**

K^{d-1} **被称为 “最次”位关键字**

实现多关键字排序

通常有两种作法:

最高位优先MSD法

最低位优先LSD法



先对K^0进行排序，并按 K^0 的不同值将记录序列分成若干子序列之后，分别对 K^1 进行排序，……，依次类推，**直至最后对最次位关键字排序完成为止。**

先对 K^{d-1} 进行排序，然后对 K^{d-2} 进行排序，依次类推，**直至对最主位关键字 K^0 排序完成为止**。

排序过程中不需要根据 “前一个” 关键字的排序结果，将记录序列分割成若干个(“前一个” 关键字不同的)子序列。

例如:学生记录含三个关键字:

系别、**班号**和**班内的序列号**，其中以系别为最主位关键字。

LSD的排序过程如下:

无序序列	3,2,30	1,2,15	3,1,20	2,3,18	2,1,20
对K^2排序	1,2,15	2,3,18	3,1,20	2,1,20	3,2,30
对K^1排序	3,1,20	2,1,20	1,2,15	3,2,30	2,3,18
对K^0排序	1,2,15	2,1,20	2,3,18	3,1,20	3,2,30

二、链式基数排序

假如多关键字的记录序列中，每个关键字的取值范围相同，则按LSD法进行排序时，可以采用**“分配-收集”**的方法，其好处是不需要进行关键字间的比较。

对于数字型或字符型的**单关键字**，可以**看成**是由多个数位或多个字符构成的**多关键字**，此时可以**采用**这种**“分配-收集”**的办法**进行排序，称作基数排序法。**

例如：对下列这组关键字

{209, 386, 768, 185, 247, 606, 230, 83

首先按其 “个位数” 取值分别为 0, 1, …, 9 **“分配”** 成 10 组，之后按从 0 至 9 的顺序将 它们 **“收集”** 在一起；

然后按其 “十位数” 取值分别为 0, 1, …, 9 **“分配”** 成 10 组，之后再按从 0 至 9 的顺序将它们 **“收集”** 在一起；

最后按其“百位数”重复一遍上述操作。

在计算机上实现基数排序时，为减少所需辅助存储空间，应采用链表作存储结构，即链式基数排序，具体作法为：

1．待排序记录以指针相链，构成一个链表；

2．“分配”时，按当前“关键字位”所取值，将记录分配到不同的“链队列”中，每个队列中记录的“关键字位”相同；

3．“收集”时，按当前关键字位取值从小到大将各队列首尾相链成一个链表；

4．对每个关键字位均重复 2) 和 3) 两步。

例如：

p→369→367→167→239→237→138→230

进行第一次分配

f[0]→230←　r[0]

f[7]→367→167→237　←　r[7]

f[8]→138←　r[8]

f[9]→369→239→139　←　r[9]

进行第一次收集

p→230→367→167→237→138→368→239→139

p→230→367→167→237→138→368→239→

进行第二次分配

f[3]→230→237→138→239→139

f[6]→367→167→368 ← r[6]

进行第二次收集

p→230→237→138→239→139→367→167→368

p→230→237→138→239→139→367→167→

进行第三次分配

f[1]→138→139→167 ← r[1]

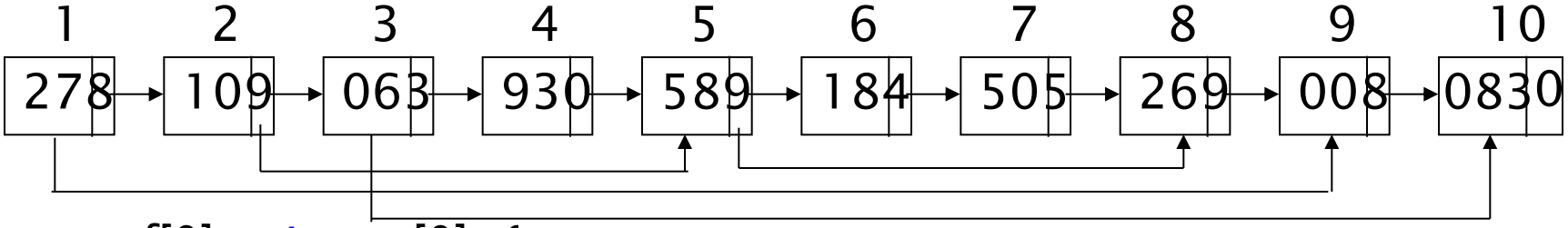
f[2]→230→237→239 ← r[2]

f[3]→367→368 ← r[3]

进行第三次收集之后便得到记录的有序序列

p→138→139→167→230→237→239→367→368

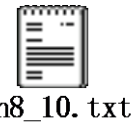
初始状态：

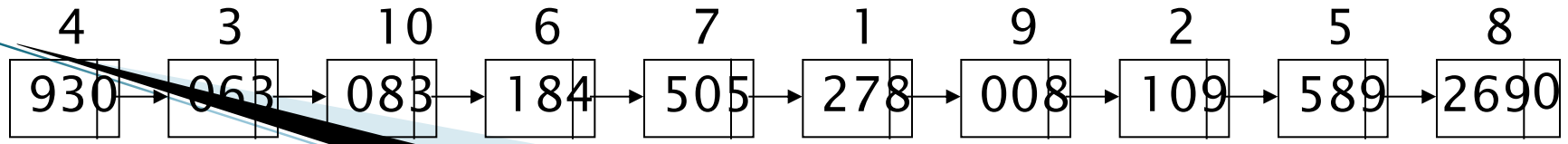


f[0]=4 e[0]=4
f[1]=
f[2]=0 e[2]=0
f[3]=0 e[3]=0
f[4]=3 e[4]=10
f[5]=
f[6]=6 e[6]=6
f[7]=7 e[7]=7
f[8]= e[8]=0
f[9]=0 e[9]=0

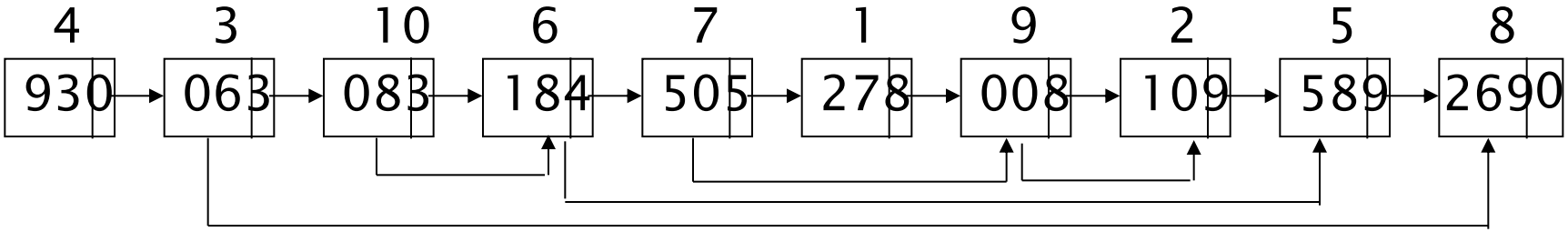
1 9

2 8



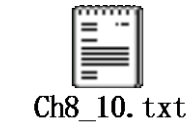


一趟收集：

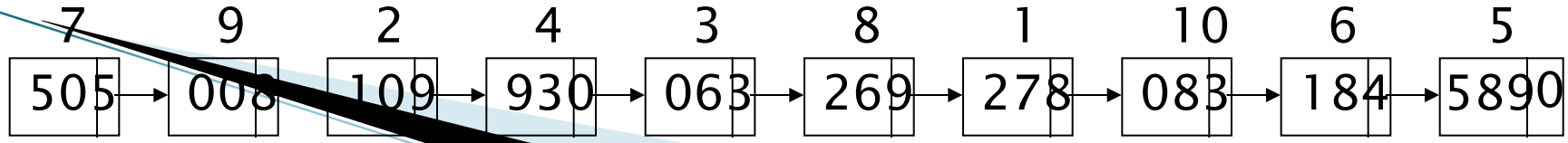


f[0]=0 7
f[1]=0
f[2]=0
f[3]=0
f[4]=0 4
f[5]=0
f[6]=0
f[7]=0
f[8]=0 3
f[9]=0 1
10

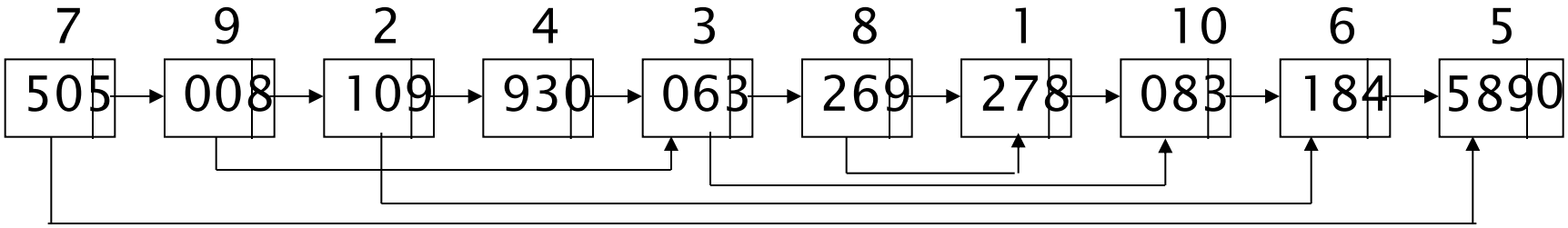
e[0]=0 2
e[1]=0
e[2]=0
e[3]=0
e[4]=0 4
e[5]=0
e[6]=0
e[7]=0
e[8]=0 8
e[9]=0 1
5



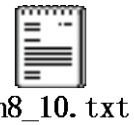
二趟收集：



二趟收集：

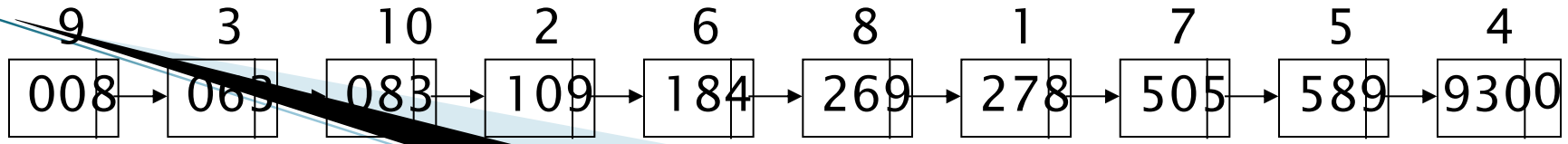


f[0]= 9　　e[0]=0 10
f[1]=　　e[1]=0
f[2]= 2　　e[2]=0 6
f[3]= 8　　e[3]=0 1
f[4]=0　　e[4]=0
f[5]=0　　e[5]=0
f[6]=0　　e[6]=0
f[7]=0 7　　e[7]=0 5
f[8]=0　　e[8]=0
f[9]=0　　e[9]=0



三趟收集：

4　　4



提醒注意：

1．“分配”和“收集”的实际操作仅为修改链表中的指针和设置队列的头、尾指针；

2．为查找使用，该链表尚需应用算法Arrange 将它调整为有序表。

链式基数排序

按当前位分配到桶

记录的关键字是非负整数。每个桶按入桶顺序收集，保持同位数字相等的记录次序。

```
#include <stddef.h>

typedef struct RadixNode {
    unsigned key;
    struct RadixNode *next;
} RadixNode;

static RadixNode *radix_pass(RadixNode *head, unsigned place)
{
    RadixNode *first[10] = {NULL}, *last[10] = {NULL};
    while (head) {
        RadixNode *node = head;
        head = head->next;
        node->next = NULL;
        unsigned digit = node->key / place % 10;
        if (last[digit]) {
            last[digit]->next = node;  // 尾插保留桶内次序
        } else {
            first[digit] = node;
        }
        last[digit] = node;
    }
```

链式基数排序

依次收集并处理下一位


```
    RadixNode dummy = {0, NULL};
    RadixNode *tail = &dummy;
    for (int digit = 0; digit < 10; ++digit) {
        if (first[digit]) {
            tail->next = first[digit];
            tail = last[digit];
        }
    }
    return dummy.next;
}

RadixNode *radix_sort(RadixNode *head)
{
    unsigned max_key = 0;
    for (RadixNode *p = head; p; p = p->next) {
        if (p->key > max_key) {
            max_key = p->key;
        }
    }
    for (unsigned place = 1; max_key / place > 0; place *= 10) {
        head = radix_pass(head, place);
        if (max_key / place < 10) {
            break;  // 最高位处理完毕，避免 place 再乘 10
        }
    }
    return head;
}
```

基数排序的时间复杂度为O(d(n+rd))

其中：分配为O(n)

收集为O(rd)(rd为“基”

d为“分配-收集”的趟数

10.7

各种排序方法的综合比较

一、时间性能

1．平均的时间性能

时间复杂度为 $O(n\log n)$：

快速排序、堆排序和归并排序

时间复杂度为 $O(n^2)$：

直接插入排序、起泡排序和简单选择排序

时间复杂度为 $O(n)$：

基数排序

2. **当待排记录序列按关键字顺序有序时**

直接插入排序和**起泡排序**能达到$O(n)$的时间复杂度，

快速排序的时间性能蜕化为$O(n^2)$。

3. **简单选择排序、堆排序和归并排序**的时间性能**不随**记录序列中关键字的分布而改变。

二、空间性能

指的是排序过程中所需的辅助空间大小

1. 所有的**简单排序方法**(包括：直接插入、起泡和简单选择) 和**堆排序**的空间复杂度**为O(1)**;

2. **快速排序为O(log*n*)**，为递归程序执行过程中，栈所需的辅助空间;

3. **归并排序**所需辅助空间最多，其空间复杂度为 O(*n*);

4. **链式基数排序**需附设队列首尾指针，则空间复杂度为 O(*rd*)。

三、排序方法的稳定性能

1. 当对多关键字的记录序列进行LSD方法排序时，必须采用稳定的排序方法。

2. 对于不稳定的排序方法，只要能举出一个实例说明即可。

例如： 对{ 4, 3, 4, 2 }进行快速排序，

得到{ 2, 3, 4, 4 }

3. **快速排序、堆排序和希尔排序是不稳定的排序方法。**



各种排序方法的比较

排序方法	平均时间	最坏情况	最好情况	辅助空间	稳定性
直接插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	√
起泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	√
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	√
快速排序	$O(n\log n)$	$O(n^2)$	$O(n\log n)$	$O(\log n)$	×
堆排序	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(1)$	×
2-路归并排序	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(n)$	√
基数排序	$O(dn)$	$O(dn)$	$O(dn)$	$O(n)$	√

排序方法的选用

- 一般情况下考虑的原则有：（1）待排序的记录个数n；（2）记录本身的大小；（3）关键字的分布情况；（4）对排序稳定性的要求等。
- 一般来说，在选择排序方法时，可有下列几种选择：

1. 若待排序的记录个数n值较小（例如n<30），则可选用插入排序法，但若记录所含数据项较多，所占存储量大时，应选用选择排序法。反之，若待排序的记录个数n值较大时，应选用快速排序法。但若待排序记录关键字有"有序"倾向时，就慎用快速排序，而宁可选用堆排序或归并排序，而后两者的最大差别是所需辅助空间不等。

2. 快速排序和归并排序在n值较小时的性能不及直接插入排序，因此在实际应用时，可将它们和插入排序"混合"使用。如在快速排序划分子区间的长度小于某值时，转而调用直接插入排序；或者对待排记录序列先逐段进行直接插入排序，然后再利用"归并操作"进行两两归并直至整个序列有序为止。

3. 基数排序的时间复杂度为O (d×n)，因此特别适合于待排记录数 n 值很大，而关键字"位数 d "较小的情况。并且还可以调整"基数"（如将基数定为100或1000等）以减少基数排序的趟数d的值。

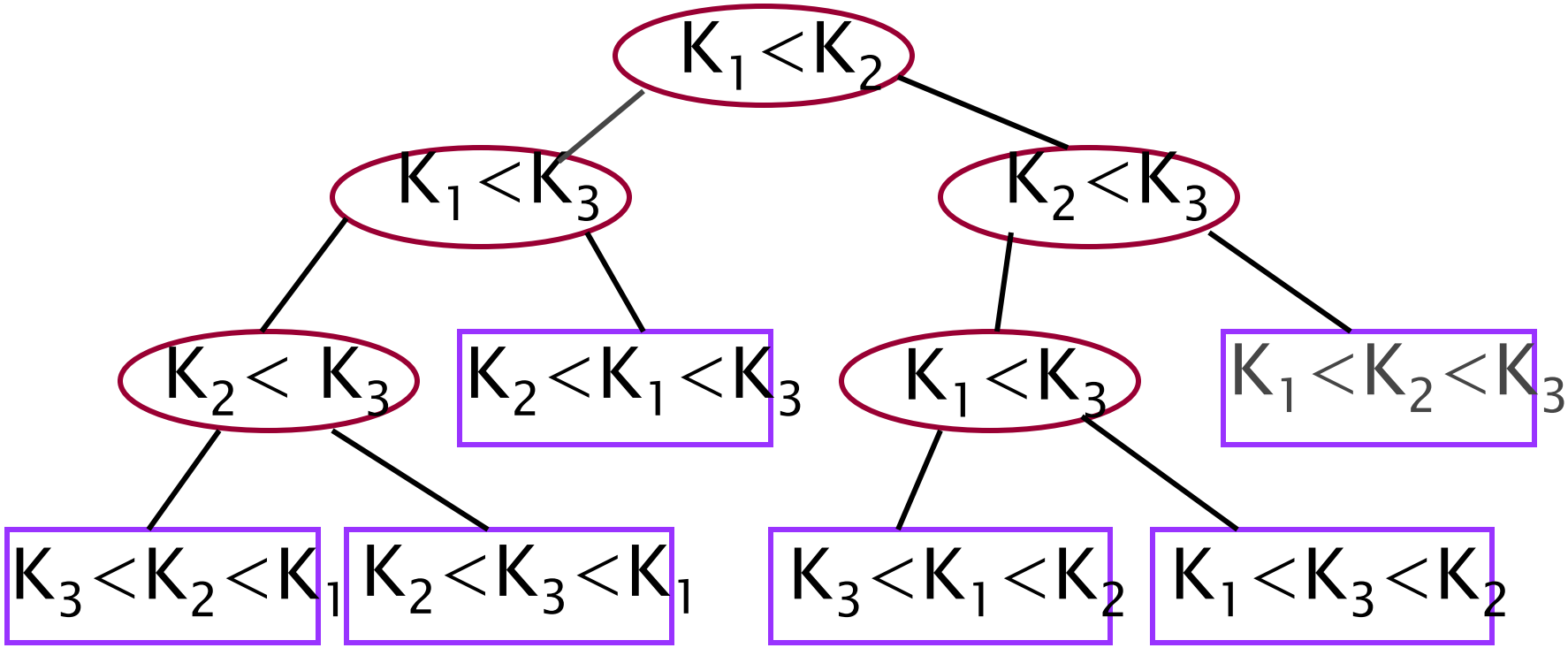
4. 一般情况下，对单关键字进行排序时，所用的排序方法是否稳定无关紧要。但当按"最次位优先"进行多关键字排序时(除第一趟外)必须选用稳定的排序方法。

四、关于“排序方法的时间复杂度的下限”

本章讨论的各种排序方法，除基数排序外，其它方法都是**基于“比较关键字”进行排序的排序方法。**

可以证明，这类排序法**可能达到的最快的时间复杂度为O($n\log n$)。**（基数排序不是基于“比较关键字”的排序方法，所以它不受这个限制。）

例如:对三个关键字进行排序的判定树如下:



1．树上的每一次“比较”都是必要的

2．树上的叶子结点包含所有可能情况。

一般情况下，对n个关键字进行排序，可能得到的结果有$n!$种，由于含$n!$个叶子结点的二叉树的深度不小于$\lceil \log_2(n!) \rceil +1$，则对 n 个关键字进行排序的比较次数至少是$\lceil \log_2(n!) \rceil \approx n\log_2 n$ (斯蒂林近似公式)。

所以，**基于“比较关键字”进行排序的**

排序方法，可能达到的最快的时间复杂

1. 了解排序的**定义**和各种排序方法的特点。熟悉各种**方法的排序过程及其依据的原则**。基于“**关键字间的比较**”进行排序的方法可以按排序过程所依据的不同原则分为**插入排序**、**交换排序**、**选择排序**、**归并排序**和计数排序等五类。

2. 掌握各种排序方法的**时间复杂度**的分析方法。能从“**关键字间的比较次数**”分析排序算法的**平均**情况和**最坏**情况的时间性能。

按平均时间复杂度划分，内部排序可分为三类：$O(n^2)$的简单排序方法，$O(n\log n)$的高效排序方法 和 $O(dn)$的基数排序方法。

3．理解排序方法“**稳定**”或“**不稳定**”的含义，弄清楚在**什么情况下**要求应用的排序方法必须是稳定的。

链式基数排序

重复关键字保持次序

输入　链表依次为 21a、12、21b。a、b 只用来区分两条关键字为 21 的记录。

预期　排序后为 12、21a、21b。两个 21 的相对次序保留。

空链表

输入　头指针为空。

预期　返回空指针。